UNIVERSIDADE PAULISTA – UNIP

CIÊNCIA DA COMPUTAÇÃO

Atividades Práticas Supervisionadas – Estrutura de Dados

DESENVOLVIMENTO DE SISTEMA PARA ANÁLISE DE PERFORMANCE DE ALGORITMOS DE ORDENAÇÃO DE DADOS UTILIZANDO IMAGENS DO CERRADO GOIANO

Alunos:

[NOME COMPLETO DO ALUNO 1] – RA: [00000000]

[NOME COMPLETO DO ALUNO 2] – RA: [00000000]

[CIDADE]

2026

ÍNDICE

1 OBJETIVO DO TRABALHO....4
2 INTRODUÇÃO....5
3 REFERENCIAL TEÓRICO....8
3.1 Conceitos fundamentais....8
3.2 Bubble Sort....8
3.3 Insertion Sort....9
3.4 Merge Sort....9
3.5 Quick Sort....10
3.6 Comparação teórica....10
4 DESENVOLVIMENTO....12
4.1 Etapas do projeto e decisões tomadas....12
4.2 Tecnologias e arquitetura....12
4.3 Obtenção dos dados....13
4.4 Armazenamento no MySQL....14
4.5 Dados externos e dados internos....15
4.6 Implementação dos algoritmos de ordenação....16
4.7 Processo de ordenação e medição do tempo....17
4.8 Listagem dos valores antes e depois da ordenação....18
4.9 Comparador de desempenho....19
4.10 Fluxo completo de uma ordenação....19
4.11 Consultas SQL do CRUD....20
4.12 Interface do usuário e ampliação de imagens....21
4.13 Análise das implementações....22
4.14 Tratamento de erros e validações....22
4.15 Execução do sistema....23
5 RESULTADOS E DISCUSSÃO....24
5.1 Metodologia dos testes....24
5.2 Dados aleatórios....25
5.3 Dados semiordenados....26
5.4 Dados ordenados....27
5.5 Dados em ordem inversa....29
5.6 Síntese e discussão....30

5.7 Expoente empírico de crescimento.....31
5.8 Confiabilidade dos resultados.....32
6 CONSIDERAÇÕES FINAIS.....33
7 REFERÊNCIAS BIBLIOGRÁFICAS.....35
8 CÓDIGO FONTE.....37
9 FICHA DE ATIVIDADES PRÁTICAS SUPERVISIONADAS.....56

1 OBJETIVO DO TRABALHO

O objetivo geral deste trabalho é desenvolver um sistema computacional completo que obtenha os dados catalogados de imagens de satélite do Cerrado Goiano, execute a ordenação desses dados com diferentes técnicas e compare o desempenho das técnicas entre si, tendo como unidade de comparação o tempo total de ordenação, sem contabilizar o tempo de aquisição dos dados.

Como objetivos específicos, o grupo se propôs a:

- obter metadados e miniaturas (thumbnails) reais de imagens de satélite do Cerrado Goiano na API pública de catálogo do Instituto Nacional de Pesquisas Espaciais (INPE) e armazená-los em um banco de dados MySQL, com a imagem gravada em campo do tipo BLOB, em quantidade superior a dez mil registros;
- implementar quatro técnicas de ordenação (Bubble Sort, Insertion Sort, Merge Sort e Quick Sort) em linguagem Python, com uma interface comum que permita ordenar os registros por qualquer um de seus campos;
- construir uma aplicação web que permita cadastrar, consultar, alterar e excluir registros, ordenar o conjunto com o algoritmo escolhido e visualizar as imagens e os valores antes e depois da ordenação;
- medir os tempos de ordenação em cenários variados (diferentes tamanhos de entrada e dados aleatórios, semiordenados, ordenados e em ordem inversa) e apresentá-los em tabelas e gráficos;
- discutir os resultados, relacionando-os à complexidade teórica de cada algoritmo e identificando vantagens e desvantagens de cada técnica em cada cenário.

2 INTRODUÇÃO

O Cerrado é o segundo maior bioma brasileiro e ocupa grande parte do território do estado de Goiás. Por sua extensão e pela pressão de atividades como a expansão agropecuária, o bioma é acompanhado de perto por programas de monitoramento por satélite, como o PRODES Cerrado, conduzido pelo INPE (INPE, 2026). O geoprocessamento de imagens orbitais permite identificar mudanças na cobertura do solo e, com isso, apoiar a fiscalização de ações que configuram crimes ambientais.

Os satélites de observação da Terra geram um volume muito grande de imagens: o enunciado desta atividade considera algo próximo de cem mil imagens de toda a região a cada vinte e quatro horas. Essas imagens são armazenadas e catalogadas em bancos de dados, em que cada registro reúne metadados (nome do arquivo, data de captura, satélite, sensor, coordenadas, tamanho) e, em muitos casos, uma versão reduzida da imagem guardada em um campo binário do tipo BLOB. Para que um analista encontre rapidamente as imagens de interesse, por exemplo as mais recentes, as de maior tamanho ou as de determinado satélite, o catálogo precisa ser ordenado, e a escolha do algoritmo de ordenação influencia diretamente o tempo de resposta do sistema.

Ordenar é um dos problemas mais estudados da Ciência da Computação. Dada uma coleção de elementos e um critério de comparação, o problema consiste em reorganizar os elementos de modo que fiquem dispostos em ordem crescente ou decrescente desse critério (CORMEN et al., 2012). Ao longo das décadas surgiram dezenas de soluções, que diferem na quantidade de operações realizadas, no uso de memória e no comportamento diante de diferentes organizações dos dados de entrada.

Entre os algoritmos mais conhecidos está o Bubble Sort (ordenação por bolha), que compara pares de elementos vizinhos e os troca quando estão fora de ordem; é simples e muito usado para ensino, embora seja pouco eficiente (ASTRACHAN, 2003). O Selection Sort (ordenação por seleção) escolhe repetidamente o menor elemento restante e o coloca na posição correta. O Insertion Sort (ordenação por inserção) constrói a sequência ordenada inserindo um elemento por vez em sua posição, como quem ordena cartas de baralho, e é eficiente para dados pequenos ou quase ordenados (SKIENA, 2008). O Shell Sort, proposto por Shell (1959), generaliza a inserção permitindo trocas entre elementos distantes. O Merge Sort (ordenação por

intercalação) divide a sequência ao meio, ordena cada metade e intercala os resultados, estratégia atribuída a John von Neumann (KNUTH, 1998). O Quick Sort, criado por Hoare (1962), particiona a sequência em torno de um pivô e ordena recursivamente as partes. O Heap Sort, de Williams (1964), utiliza uma estrutura de dados chamada heap para extrair sempre o maior elemento. Existem ainda algoritmos que não se baseiam em comparações, como o Counting Sort e o Radix Sort, aplicáveis a chaves inteiras de faixa limitada, e híbridos modernos, como o Timsort, adotado pela linguagem Python (PYTHON SOFTWARE FOUNDATION, 2026).

Para comparar algoritmos de ordenação há duas abordagens complementares. A análise teórica estima, em função do tamanho n da entrada, quantas operações cada algoritmo executa, e permite classificá-los em famílias de custo. A análise empírica, por sua vez, implementa os algoritmos e mede o tempo gasto em execuções reais, o que revela efeitos que a teoria omite: o custo de cada operação na linguagem escolhida, o uso de memória, o comportamento diante de dados repetidos e a influência da organização inicial dos dados. Este trabalho combina as duas abordagens: a seção 3 apresenta a análise teórica, e as seções 4 e 5 descrevem e discutem as medições feitas no sistema desenvolvido.

O armazenamento das imagens no próprio banco de dados também merece destaque. Os bancos relacionais oferecem o tipo BLOB (Binary Large Object), que guarda sequências de bytes de qualquer conteúdo, como imagens, ao lado dos demais campos do registro (ELMASRI; NAVATHE, 2019). Essa escolha mantém metadados e imagem sob a mesma garantia de integridade e permite consultas combinadas, mas exige cuidado: ler as imagens junto com os metadados em operações que não precisam delas, como a ordenação, desperdiçaria tempo e memória. Esse cuidado orientou várias decisões descritas na seção 4.

Neste trabalho, o grupo selecionou quatro técnicas de naturezas distintas: duas quadráticas, simples e de comportamento bastante sensível à organização dos dados (Bubble Sort e Insertion Sort), e duas baseadas em divisão e conquista, com custo médio proporcional a n log n (Merge Sort e Quick Sort). A escolha permite observar, em um mesmo sistema, a diferença prática entre classes de complexidade e o efeito de dados já ordenados, semiordenados, aleatórios e invertidos sobre cada técnica.

Para obter dados reais, o sistema consulta o catálogo STAC (SpatioTemporal Asset Catalog) do projeto Brazil Data Cube, mantido pelo INPE, e armazena em um banco MySQL os metadados e as miniaturas de milhares de cenas do Cerrado Goiano

captadas pelos satélites Sentinel-2, CBERS-4 e Amazonia-1. Sobre esse conjunto, uma aplicação web desenvolvida em Python com o framework Flask permite operar o catálogo, ordenar seus registros e comparar o desempenho dos algoritmos.

Este documento está organizado da seguinte forma: a seção 3 apresenta o referencial teórico sobre os algoritmos escolhidos; a seção 4 descreve o desenvolvimento do sistema, desde a obtenção dos dados até a apresentação dos resultados comparativos; a seção 5 expõe os testes realizados e discute os resultados; a seção 6 traz as considerações finais; por fim, são listadas as referências bibliográficas e apresentado o código-fonte completo.

3 REFERENCIAL TEÓRICO

3.1 Conceitos fundamentais

A análise de um algoritmo de ordenação costuma considerar três propriedades. A primeira é a complexidade de tempo, expressa em notação assintótica em função do número n de elementos: $O(n^2)$ indica que o número de operações cresce com o quadrado de n, e $O(n \log n)$ indica crescimento bem mais lento (CORMEN et al., 2012; SZWARCFITER; MARKENZON, 2010). A segunda é o uso de memória auxiliar: um algoritmo é dito in-place quando reorganiza os elementos na própria sequência, usando quantidade de memória adicional constante ou pequena. A terceira é a estabilidade: um algoritmo é estável quando elementos com chaves iguais mantêm a ordem relativa que tinham na entrada (KNUTH, 1998). A estabilidade importa quando se ordena por mais de um critério em sequência, como ordenar o catálogo por satélite e, depois, por data.

Os quatro algoritmos escolhidos são baseados em comparação: descobrem a ordem exclusivamente comparando pares de chaves. Para esse tipo de algoritmo, sabe-se que nenhum pode ter pior caso melhor que $\Omega(n \log n)$ comparações (CORMEN et al., 2012), o que torna Merge Sort e Quick Sort assintoticamente ótimos no caso médio.

3.2 Bubble Sort

O Bubble Sort percorre a sequência repetidas vezes. Em cada passada compara cada elemento com o seguinte e os troca se estiverem fora de ordem; assim, ao final da primeira passada o maior elemento já está na última posição, e a cada passada o maior elemento restante “sobe” até seu lugar, como uma bolha. No pior caso e no caso médio realiza da ordem de $n^2/2$ comparações, sendo $O(n^2)$. Com a otimização de interromper o processo quando uma passada completa não realiza nenhuma troca, o melhor caso (sequência já ordenada) passa a ser $O(n)$. É estável e in-place (ZIVIANI, 2007; ASTRACHAN, 2003).

O pseudocódigo a seguir resume o algoritmo, em que A é a sequência e n seu tamanho:

```
BOLHA(A, n)
  para i de 0 até n-2:
    trocou <- falso
    para j de 0 até n-2-i:
      se A[j] > A[j+1]:
        troca A[j] com A[j+1]
```

```
        trocou <- verdadeiro
    se não trocou: pare
```

Para a sequência [5, 2, 4, 1], a primeira passada produz [2, 4, 1, 5]; a segunda, [2, 1, 4, 5]; a terceira, [1, 2, 4, 5], e a quarta passada, sem trocas, encerra o processo.

3.3 Insertion Sort

O Insertion Sort mantém uma parte inicial da sequência já ordenada e, a cada passo, retira o próximo elemento e o desloca para a esquerda até encontrar sua posição entre os elementos já ordenados. O custo é proporcional ao número de inversões da entrada, isto é, de pares de elementos fora de ordem. Em sequências invertidas esse número é da ordem de $n^2/2$, e o algoritmo é $O(n^2)$; em sequências já ordenadas não há deslocamentos, e o custo é O(n); em sequências quase ordenadas o custo é próximo do linear (SEDGEWICK; WAYNE, 2011). É estável e in-place, e costuma ser mais rápido que o Bubble Sort em entradas aleatórias, pois realiza menos trocas.

```
INSERCAO(A, n)
  para i de 1 até n-1:
    atual <- A[i]
    j <- i - 1
    enquanto j >= 0 e A[j] > atual:
      A[j+1] <- A[j]
      j <- j - 1
    A[j+1] <- atual
```

Para [5, 2, 4, 1]: o 2 é inserido antes do 5 ([2, 5, 4, 1]); o 4 fica entre o 2 e o 5 ([2, 4, 5, 1]); e o 1 desloca todos os demais, resultando em [1, 2, 4, 5].

3.4 Merge Sort

O Merge Sort aplica o paradigma de divisão e conquista: divide a sequência em duas metades, ordena cada metade recursivamente e as intercala em uma única sequência ordenada. A intercalação percorre as duas metades com dois índices e copia sempre o menor dos elementos apontados, o que custa tempo linear. Como a divisão ao meio produz cerca de $\log_2 n$ níveis de recursão e cada nível custa O(n), o algoritmo é O(n log n) no melhor, no médio e no pior caso (CORMEN et al., 2012). Em contrapartida, exige memória auxiliar proporcional a n para armazenar as sequências intermediárias. Quando a intercalação usa a comparação “menor ou igual”, o algoritmo é estável. Essa mesma ideia de intercalar blocos já ordenados é a base da ordenação externa, usada para dados maiores que a memória disponível (KNUTH, 1998).

```
MERGESORT(A)
  se tamanho(A) <= 1: devolva A
```

```
  E <- MERGESORT(primeira metade de A)
  D <- MERGESORT(segunda metade de A)
  devolva INTERCALAR(E, D)

INTERCALAR(E, D)
  enquanto E e D têm elementos:
    copie para a saída o menor entre os primeiros de E e D
      (em empate, o de E)
  copie o que sobrou de E ou de D
```

3.5 Quick Sort

O Quick Sort escolhe um elemento chamado pivô e particiona a sequência em elementos menores, iguais e maiores que ele; em seguida ordena recursivamente as partes menor e maior, e o resultado é a concatenação das três partes (HOARE, 1962). Se o pivô divide a sequência de forma equilibrada, são necessários cerca de $\log_2 n$ níveis de recursão e o custo é O(n log n); se o pivô for sempre o menor ou o maior elemento, a recursão tem n níveis e o custo degenera para $O(n^2)$. Por isso a escolha do pivô é decisiva: pivôs fixos nas pontas pioram em dados já ordenados, e estratégias como o elemento do meio ou a mediana de três reduzem o risco (SEDGEWICK, 1978). Na formulação clássica o algoritmo é in-place e instável; na formulação funcional com três listas, adotada neste trabalho, usa memória auxiliar, mas trata bem chaves repetidas, pois todos os elementos iguais ao pivô são colocados de uma vez em seu lugar definitivo.

```
QUICKSORT(A)
  se tamanho(A) <= 1: devolva A
  pivô <- A[meio]
  menores <- elementos de A menores que o pivô
  iguais  <- elementos de A iguais ao pivô
  maiores <- elementos de A maiores que o pivô
  devolva QUICKSORT(menores) + iguais + QUICKSORT(maiores)
```

3.6 Comparação teórica

A Tabela 1 resume as características teóricas das quatro técnicas, que serão confrontadas com as medições na seção 5.

Tabela 1 – Características teóricas dos algoritmos escolhidos

Algoritmo	Melhor caso	Caso médio	Pior caso	Memória extra	Estável
Bubble Sort	O(n)*	$O(n^2)$	$O(n^2)$	O(1)	Sim
Insertion Sort	O(n)	$O(n^2)$	$O(n^2)$	O(1)	Sim
Merge Sort	O(n log n)	O(n log n)	O(n log n)	O(n)	Sim
Quick Sort	O(n log n)	O(n log n)	$O(n^2)$	O(log n) a O(n)	Não**

Fonte: adaptado de Cormen et al. (2012) e Knuth (1998). * Com interrupção antecipada. ** Na versão in-place; a versão com listas usada no trabalho preserva a ordem relativa apenas dentro do bloco de elementos iguais.

4 DESENVOLVIMENTO

4.1 Etapas do projeto e decisões tomadas

O desenvolvimento ocorreu de forma incremental. Em uma primeira versão, o grupo construiu um sistema completo com banco SQLite, cinco algoritmos (incluindo o Radix Sort), geração de dados sintéticos e suíte de testes automatizados. Após conversa com o professor, duas decisões mudaram o rumo do projeto. A primeira foi a troca do SQLite pelo MySQL, administrado pelo phpMyAdmin, por ser o sistema gerenciador de banco de dados adotado na disciplina. A segunda foi a simplificação do escopo: restringir os algoritmos aos quatro apresentados neste trabalho, que são os que o grupo consegue explicar integralmente, e operar somente sobre imagens reais, retirando a geração de registros sintéticos do sistema.

As etapas executadas, em ordem cronológica, foram: (1) definição do tema, da linguagem (Python) e das tecnologias; (2) estudo da API do INPE e definição dos filtros para obter apenas cenas do Cerrado Goiano; (3) modelagem da tabela e implementação do acesso ao banco; (4) implementação dos quatro algoritmos com interface comum; (5) construção da aplicação web com as páginas de ordenação, de cadastro (CRUD) e de comparação; (6) carga dos dados reais e migração do SQLite para o MySQL; (7) ajustes de interface solicitados na revisão, como a ampliação da imagem ao clicar, a busca por identificador e a troca de imagem na edição; e (8) medição do desempenho e redação desta monografia.

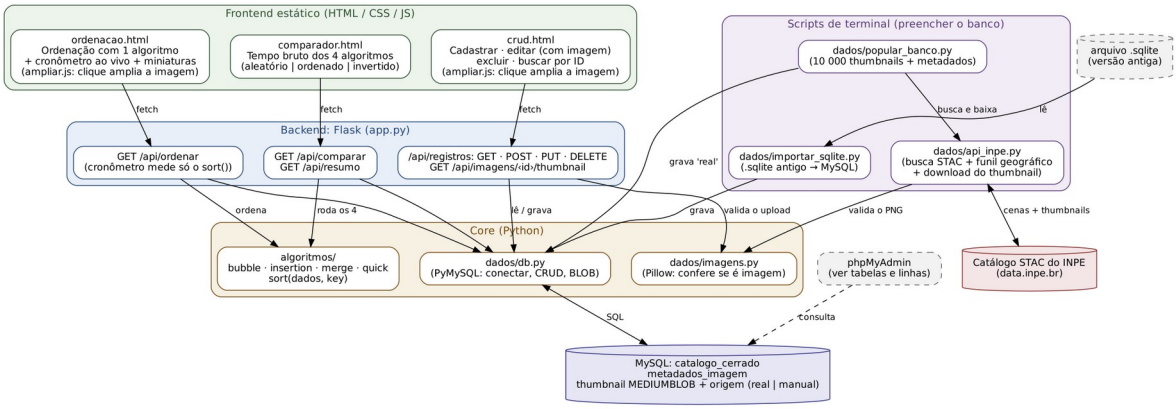
4.2 Tecnologias e arquitetura

O sistema foi escrito em Python. O servidor web usa o framework Flask, que associa funções Python a endereços (rotas) e devolve páginas e dados em formato JSON (GRINBERG, 2018; PALLETS PROJECTS, 2026). O acesso ao MySQL é feito pela biblioteca PyMySQL (PYMYSQL, 2026); a biblioteca Requests faz as chamadas à API do INPE; a biblioteca Pillow identifica o formato dos arquivos de imagem enviados pelos usuários. A interface é composta por três páginas HTML estáticas, com JavaScript puro para chamar a API do servidor, e uma folha de estilos CSS.

A arquitetura, ilustrada na Figura 1, separa responsabilidades em camadas. As páginas do navegador chamam as rotas do arquivo app.py; este usa o pacote core.dados para ler e gravar no MySQL e o pacote core.algoritmos para ordenar. A

obtenção dos dados do INPE ocorre fora do servidor, em um script de carga executado uma única vez.

Figura 1 – Arquitetura do sistema



Fonte: elaborada pelos autores.

4.3 Obtenção dos dados

Os dados externos vêm do catálogo STAC do Brazil Data Cube (BRAZIL DATA CUBE, 2026), acessível pelo endereço https://data.inpe.br/bdc/stac/v1. O STAC é uma especificação aberta (SPATIOTEMPORAL ASSET CATALOG, 2026) para descrever coleções de dados geoespaciais; cada cena é devolvida como um item em JSON contendo identificador, caixa envolvente (bbox), propriedades como data e plataforma, e uma lista de arquivos associados (assets). Entre eles, existe um asset chamado thumbnail, uma imagem PNG de pequena dimensão. O arquivo completo da cena (asset de produto) tem cerca de um gigabyte e jamais é baixado; o sistema usa apenas a miniatura, o que torna viável armazenar dez mil imagens em um banco de dados.

A consulta é feita na rota de busca (/search) com os parâmetros coleção, caixa envolvente do Cerrado Goiano, intervalo de datas e limite de 500 itens por página. A caixa adotada vai de 50,5° a 46,0° de longitude oeste e de 19,0° a 13,0° de latitude sul. Como o catálogo devolve páginas de resultados, o código segue o link de rel igual a “next” de cada resposta até esgotar os resultados. O trecho a seguir, extraído do módulo api_inpe.py, mostra esse laço:

```
def _buscar_colecao(colecao, bbox, data_inicio, data_fim, maximo):
    """Busca UMA coleção, seguindo o link 'next' de cada página."""
    url = f"{STAC_BASE_URL}/search"
    params = {
        "collections": colecao,  # sem isso vêm cenas de qualquer coleção (radar,
GOES...)
        "bbox": ",".join(str(c) for c in bbox),
```

```
        "datetime": f"{data_inicio}T00:00:00Z/{data_fim}T23:59:59Z",
        "limit": 500,
    }
    aceitos, vistos = [], set()
    while url and len(aceitos) < maximo:
        resp = SESSAO.get(url, params=params, timeout=30)
        resp.raise_for_status()
        corpo = resp.json()
        features = corpo.get("features", [])
        if not features:
            break
        for item in features:
            if item.get("id") not in vistos and cena_pertence_ao_cerrado(item, bbox):
                vistos.add(item.get("id"))
                aceitos.append(item)
        url = next((l["href"] for l in corpo.get("links", []) if l.get("rel") ==
"next"), None)
        params = None  # o link 'next' já traz os filtros
    return aceitos[:maximo]
```

Por si só a caixa envolvente não basta, pois algumas coleções contêm produtos que cobrem continentes inteiros e “tocam” o Cerrado. Por isso o sistema aplica um funil: (a) usa apenas as coleções S2_L1C_BUNDLE-1 (Sentinel-2, sensor MSI), CB4-MUX-L4-DN-1 (CBERS-4, sensor MUX) e AMZ1-WFI-L2-DN-1 (Amazonia-1, sensor WFI), que possuem miniatura em PNG; (b) descarta cenas cujo lado excede 12°; e (c) mantém somente cenas cujo centro está dentro da caixa do Cerrado Goiano. A função cena_pertence_ao_cerrado implementa os itens (b) e (c). As coleções são consultadas em sequência, passando-se à seguinte somente se ainda faltarem cenas para completar a quantidade desejada.

O script popular_banco.py pede ao INPE cerca de 20% mais cenas do que o necessário, porque algumas não possuem miniatura baixável, e para cada cena baixa o PNG, extrai os metadados e grava o registro no MySQL, confirmando a transação a cada 500 registros. A meta é de dez mil registros reais. O tempo gasto nessa carga não entra em nenhuma medição de desempenho, conforme exige o enunciado.

4.4 Armazenamento no MySQL

Seguindo o modelo relacional (ELMASRI; NAVATHE, 2019), os dados catalogados ficam em uma única tabela, chamada metadados_imagem, criada automaticamente pelo sistema na primeira conexão. A miniatura é gravada na coluna thumbnail_blob, do tipo MEDIUMBLOB, que comporta até 16 MB (o tipo BLOB comum guarda no máximo 64 KB, limite que miniaturas maiores poderiam ultrapassar; ORACLE, 2026). A coluna origem usa o tipo ENUM, que só aceita os valores “real” (registro vindo do INPE) e “manual” (registro cadastrado pelo usuário no CRUD). O comando de criação é mostrado a seguir:

```
SCHEMA = """
```

```
CREATE TABLE IF NOT EXISTS metadados_imagem (
    id              INT AUTO_INCREMENT PRIMARY KEY,
    nome_arquivo    VARCHAR(255) NOT NULL,
    data_captura    VARCHAR(40),
    satelite        VARCHAR(100),
    sensor          VARCHAR(100),
    latitude        DOUBLE,
    longitude       DOUBLE,
    tamanho_bytes   INT NOT NULL,
    thumbnail_blob  MEDIUMBLOB NOT NULL,
    origem          ENUM('real', 'manual') NOT NULL,
    criado_em       TIMESTAMP DEFAULT CURRENT_TIMESTAMP
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4
"""
```

O identificador id é numerado pelo próprio MySQL (AUTO_INCREMENT) e é a ordem de inserção dos registros. O acesso é feito pela função conectar, um gerenciador de contexto que abre a conexão, cria o banco e a tabela caso não existam, confirma a transação (commit) ao final ou a desfaz (rollback) em caso de erro, e sempre fecha a conexão. Todos os comandos SQL com valores informados pelo usuário usam parâmetros (%s) em vez de concatenação de texto, o que impede ataques de injeção de SQL.

Uma decisão de projeto importante foi separar a leitura dos metadados da leitura das imagens. A função listar_todos executa um SELECT que devolve todas as colunas, exceto o BLOB. Como cada miniatura ocupa dezenas ou centenas de quilobytes, carregar dez mil imagens para a memória apenas para ordenar encareceria a leitura e consumiria memória sem benefício, já que a ordenação compara somente campos como o tamanho ou a data. As imagens são lidas sob demanda, uma a uma, pela rota /api/imagens/<id>/thumbnail, apenas para as linhas exibidas na tela. A coluna tamanho_bytes, calculada na gravação a partir do tamanho do PNG, permite ordenar por tamanho sem ler o BLOB.

```
def listar_todos(conn) -> list[dict]:
    """SELECT: todas as linhas (sem o BLOB), como lista de dicionários. É isto que vai
para os algoritmos."""
    cur = _executar(conn, f"SELECT {COLUNAS} FROM metadados_imagem")
    return cur.fetchall()
```

4.5 Dados externos e dados internos

O enunciado exige que as ordenações usem dados externos, obtidos de valores previamente armazenados, e dados internos, valores gerados pelo programa ou fornecidos pelo usuário. No sistema, os dados externos são os registros reais do INPE, previamente armazenados no banco de dados. Os dados fornecidos pelo usuário são os registros cadastrados na página CRUD, em que o usuário informa nome, satélite, sensor, data e coordenadas e envia uma imagem; o sistema valida a

imagem com a biblioteca Pillow, calcula seu tamanho e grava a linha com origem “manual”. Os dados gerados pelo programa são os registros com chaves aleatórias criados pelo script de medição benchmark/medir_desempenho.py, que produz entradas de 100 a 10.000 elementos nos quatro cenários de teste da seção 5.

4.6 Implementação dos algoritmos de ordenação

Os quatro algoritmos ficam no pacote core.algoritmos, um arquivo para cada técnica, e todos expõem a mesma interface: sort(dados, key), em que dados é a lista de registros (dicionários) e key é uma função que extrai de cada registro o valor a ser comparado. Assim, o mesmo código ordena por tamanho, data, nome, satélite ou identificador. Todos devolvem uma nova lista e não alteram a lista original, de modo que cada algoritmo recebe exatamente os mesmos dados de entrada nos testes comparativos.

O Bubble Sort foi implementado com a interrupção antecipada: se uma passada completa não faz trocas, a lista já está ordenada e o laço termina.

```
def sort(dados: list, key=lambda x: x) -> list:
    lista = dados.copy()
    n = len(lista)

    for i in range(n - 1):
        trocou = False
        for j in range(n - 1 - i):
            if key(lista[j]) > key(lista[j + 1]):
                lista[j], lista[j + 1] = lista[j + 1], lista[j]
                trocou = True
        if not trocou:
            break  # já está ordenada, não precisa das passadas restantes

    return lista
```

No Insertion Sort, cada elemento é guardado em uma variável, os elementos maiores à sua esquerda são deslocados uma posição para a direita e o elemento é colocado na posição aberta.

```
def sort(dados: list, key=lambda x: x) -> list:
    lista = dados.copy()

    for i in range(1, len(lista)):
        atual = lista[i]
        chave_atual = key(atual)
        j = i - 1

        # desloca os elementos maiores que `atual` uma posição à direita
        while j >= 0 and key(lista[j]) > chave_atual:
            lista[j + 1] = lista[j]
            j -= 1

        lista[j + 1] = atual

    return lista
```

O Merge Sort divide a lista ao meio, ordena as metades por chamadas recursivas e as combina com a função _intercalar. O uso de “<=” na comparação garante a estabilidade, pois, em caso de empate, o elemento da metade esquerda (que veio antes) é copiado primeiro.

```
def sort(dados: list, key=lambda x: x) -> list:
    lista = dados.copy()

    if len(lista) <= 1:
        return lista

    meio = len(lista) // 2
    esquerda = sort(lista[:meio], key)
    direita = sort(lista[meio:], key)

    return _intercalar(esquerda, direita, key)


def _intercalar(esquerda: list, direita: list, key) -> list:
    """Combina duas listas já ordenadas em uma só, ordenada."""
    resultado = []
    i = j = 0

    while i < len(esquerda) and j < len(direita):
        # <= (não <) garante estabilidade: em caso de empate, o elemento
        # da esquerda (que veio antes na lista original) entra primeiro
        if key(esquerda[i]) <= key(direita[j]):
            resultado.append(esquerda[i])
            i += 1
        else:
            resultado.append(direita[j])
            j += 1

    resultado.extend(esquerda[i:])
    resultado.extend(direita[j:])
    return resultado
```

O Quick Sort usa como pivô o elemento do meio da lista e separa os registros em três listas (menores, iguais e maiores). A escolha do elemento central evita o pior caso em listas já ordenadas ou invertidas, e a lista de iguais evita recursão desnecessária quando há chaves repetidas, situação comum, pois várias miniaturas podem ter o mesmo tamanho em bytes.

```
def sort(dados: list, key=lambda x: x) -> list:
    if len(dados) <= 1:
        return dados.copy()

    pivo = dados[len(dados) // 2]
    chave_pivo = key(pivo)

    menores = [x for x in dados if key(x) < chave_pivo]
    iguais = [x for x in dados if key(x) == chave_pivo]
    maiores = [x for x in dados if key(x) > chave_pivo]

    return sort(menores, key) + iguais + sort(maiores, key)
```

4.7 Processo de ordenação e medição do tempo

A ordenação ocorre na rota /api/ordenar. O servidor valida o algoritmo e o campo recebidos, lê os registros do banco, ordena e devolve o resultado em JSON. A

função _chave monta a função key adequada ao campo: numérica para tamanho_bytes e id, e textual para os demais (valores ausentes viram texto vazio, para que a comparação nunca falhe). O ponto mais importante é o posicionamento do cronômetro: a leitura do banco ocorre antes, e o cronômetro envolve somente a chamada ao algoritmo, em atendimento ao enunciado, que não admite contabilizar o tempo de aquisição dos dados.

```
def ordenar():
    algoritmo = request.args.get("algoritmo", "quick")
    campo = request.args.get("campo", "tamanho_bytes")
    _validar_ordenacao(algoritmo, campo)

    with _banco() as conn:
        registros = db.listar_todos(conn)

    inicio = time.perf_counter()  # o cronômetro mede SÓ o sort(), sem a leitura do
banco
    ordenados = ALGORITMOS[algoritmo][1].sort(registros, key=_chave(campo))
    tempo = time.perf_counter() - inicio

    return jsonify({
        "algoritmo": ALGORITMOS[algoritmo][0],
        "campo": campo,
        "quantidade": len(ordenados),
        "tempo_segundos": tempo,
        "itens": ordenados,
    })
```

A medição usa time.perf_counter, o relógio de maior resolução disponível em Python, apropriado para medir intervalos curtos (PYTHON SOFTWARE FOUNDATION, 2026). Na página de ordenação, além do tempo do algoritmo, a interface mostra um cronômetro ao vivo, atualizado a cada 50 ms por JavaScript, que inclui o tempo de ida e volta ao servidor; os dois valores são exibidos separadamente para deixar claro que apenas o primeiro é o tempo de ordenação.

4.8 Listagem dos valores antes e depois da ordenação

A página CRUD lista os registros na ordem de inserção, do mais recente para o mais antigo (identificador decrescente), com miniatura, nome, satélite, sensor, data, tamanho e origem, paginados de 20 em 20. Essa listagem representa o estado dos dados antes da ordenação. A página de ordenação exibe, após cada execução, a lista completa já ordenada, com as mesmas colunas e a mesma paginação local, o que permite conferir os valores depois da ordenação. Ao clicar em qualquer miniatura, uma camada de sobreposição mostra a imagem ampliada, que se fecha com um clique ou com a tecla Esc. Ordenar por id restabelece a ordem de inserção original, o que facilita a comparação.

A página CRUD também oferece busca de um registro por identificador, cadastro com imagem, edição dos campos com substituição opcional da imagem e exclusão. Para evitar que o navegador exiba uma miniatura antiga após a troca da imagem, a rota de miniaturas envia o cabeçalho Cache-Control: no-cache e a interface acrescenta um número de versão ao endereço da imagem.

4.9 Comparador de desempenho

A terceira página da aplicação executa os quatro algoritmos, um após o outro, sobre os mesmos dados e mostra o tempo de cada um. Os dados são uma amostra aleatória do banco (função random.sample) com a quantidade escolhida pelo usuário, e o cenário pode ser aleatório, ordenado ou invertido. Para os dois últimos, a amostra é ordenada antes de iniciar o cronômetro, de modo que o tempo de preparação não é medido. O resultado é apresentado em linhas simples, no formato “Algoritmo: tempo em segundos”.

```
    resultados = []
    for nome, (rotulo, modulo) in ALGORITMOS.items():
        inicio = time.perf_counter()
        modulo.sort(amostra, key=chave)
        resultados.append({"algoritmo": rotulo, "tempo_segundos": time.perf_counter() -
inicio})
```

Para as tabelas e gráficos desta monografia, o grupo preferiu um procedimento mais rigoroso do que uma única execução, descrito na seção 5: um script próprio repete cada medição e adota a mediana dos tempos, e verifica a correção de cada resultado.

4.10 Fluxo completo de uma ordenação

Para esclarecer como as partes se encaixam, descreve-se o caminho completo de uma ordenação, desde o clique do usuário até a exibição do resultado. (1) O usuário escolhe o campo e o algoritmo na página de ordenação e clica no botão. (2) O JavaScript da página inicia o cronômetro ao vivo e envia, com a função fetch, uma requisição GET para /api/ordenar com os parâmetros escolhidos. (3) O Flask chama a função ordenar, que valida os parâmetros e abre uma conexão com o MySQL. (4) A função listar_todos lê os metadados de todos os registros, sem as imagens. (5) O cronômetro do servidor é disparado, o algoritmo ordena a lista e o cronômetro é lido logo em seguida. (6) O servidor responde com JSON contendo o nome do algoritmo, o campo, a quantidade, o tempo da ordenação e a lista ordenada. (7) O navegador

interrompe o cronômetro ao vivo, mostra o tempo recebido e desenha a primeira página de vinte linhas, pedindo ao servidor apenas as imagens dessas vinte linhas. O trecho do JavaScript que realiza os passos (2), (6) e (7) é mostrado a seguir:

```
        const resposta = await fetch("/api/ordenar?" + params);
        const dados = await resposta.json();
        if (!resposta.ok) throw new Error(dados.erro || "erro no servidor");
        itens = dados.itens;
        pagina = 1;
        $("tempo-sort").textContent = formatarTempo(dados.tempo_segundos);
        $("status").textContent = `${dados.quantidade} registros ordenados com $
{dados.algoritmo}.`;
        desenharPagina();
```

A paginação da lista ordenada é feita no navegador, com o método slice sobre a lista recebida, de modo que trocar de página não exige nova ordenação nem nova consulta ao banco. Já as miniaturas são carregadas de forma preguiçosa (atributo loading igual a “lazy”), isto é, o navegador só as solicita quando a linha se aproxima da área visível.

4.11 Consultas SQL do CRUD

As operações de cadastro, consulta, alteração e exclusão correspondem aos comandos SQL INSERT, SELECT, UPDATE e DELETE. A listagem paginada usa a cláusula ORDER BY id DESC, que ordena do identificador maior para o menor (os registros mais novos primeiro), combinada com LIMIT, que limita a quantidade de linhas devolvidas, e OFFSET, que indica quantas linhas pular. Para a página 3 com 20 linhas por página, por exemplo, o banco pula 40 linhas e devolve as 20 seguintes. A função também devolve o total de registros, calculado por SELECT COUNT(*), para que a interface saiba quantas páginas existem.

```
def listar_pagina(conn, pagina: int, por_pagina: int):
    """Uma página de linhas, as mais novas primeiro. LIMIT = quantas; OFFSET = quantas
pular."""
    cur = _executar(
        conn,
        f"SELECT {COLUNAS} FROM metadados_imagem ORDER BY id DESC LIMIT %s OFFSET %s",
        (por_pagina, (pagina - 1) * por_pagina),
    )
    return cur.fetchall(), contar(conn)
```

A alteração de registros monta o comando UPDATE dinamicamente, apenas com as colunas que o usuário enviou. Os nomes das colunas vêm de uma lista fixa (CAMPOS_EDITAVEIS), nunca do texto digitado, e os valores seguem como parâmetros. Se o usuário enviar uma nova imagem, a função também troca a coluna thumbnail_blob e recalcula tamanho_bytes, mantendo coerente o campo usado como

chave de ordenação. A função devolve falso quando o identificador não existe, o que a rota converte em resposta 404.

```
def atualizar(conn, id_: int, campos: dict, imagem: bytes | None = None) -> bool:
    """
    UPDATE ... SET ... WHERE id = %s: muda só os campos informados. False se o id não
existe.
    Se vier uma imagem nova, troca também o thumbnail_blob e recalcula o tamanho_bytes.
    """
    campos = {k: v for k, v in campos.items() if k in CAMPOS_EDITAVEIS}
    if imagem is not None:
        campos["thumbnail_blob"] = imagem
        campos["tamanho_bytes"] = len(imagem)
    if campos:
        trecho = ", ".join(f"{k} = %s" for k in campos)  # nomes vêm de lista fixa,
nunca do usuário
        _executar(conn, f"UPDATE metadados_imagem SET {trecho} WHERE id = %s",
(*campos.values(), id_))
    return obter(conn, id_) is not None
```

4.12 Interface do usuário e ampliação de imagens

A interface tem três páginas, acessíveis por uma barra de navegação: Ordenação, CRUD e Comparador. Foram escritas em HTML e JavaScript puros, sem bibliotecas externas, e compartilham a folha de estilos estilo.css, o que mantém o projeto pequeno e fácil de explicar. A página Ordenação reúne os seletores de campo e de algoritmo, o botão, os dois tempos (ao vivo e da ordenação), e a tabela paginada. A página CRUD reúne o formulário de cadastro e edição, o campo de busca por identificador e a tabela de registros com os botões de editar e excluir. A página Comparador reúne a escolha de campo, cenário e quantidade, e o resultado em linhas de texto.

A visualização ampliada das imagens é feita pelo arquivo ampliar.js, carregado em todas as páginas. O script cria uma camada que cobre a tela e contém uma imagem grande, inicialmente oculta. Em vez de associar um tratamento de clique a cada miniatura, ele usa delegação de eventos: um único ouvinte no documento inteiro verifica, com o método closest, se o clique ocorreu sobre uma imagem dentro de uma célula de tabela. Essa técnica funciona também para miniaturas criadas dinamicamente a cada troca de página.

```
  document.addEventListener("click", (e) => {
    const miniatura = e.target.closest("td img");
    if (miniatura) {
      grande.src = miniatura.src;
      fundo.hidden = false;
    } else if (!fundo.hidden) {
      fechar();
    }
  });
```

4.13 Análise das implementações

Antes dos testes, é útil estimar o trabalho executado por cada implementação. No Bubble Sort, o número de comparações no pior caso é n(n-1)/2, ou seja, quase cinquenta milhões de comparações para dez mil registros, cada uma com duas chamadas à função de chave. No Insertion Sort, o número de comparações é próximo ao de deslocamentos mais n, e depende do número de inversões da lista: zero para uma lista ordenada e n(n-1)/2 para uma lista invertida. No Merge Sort, o trabalho é de cerca de $n \cdot \log_2 n$ comparações (para dez mil elementos, aproximadamente 133 mil), mais a criação de listas intermediárias. No Quick Sort, o trabalho por nível de recursão é de três percursos de n elementos, com uma chamada à função de chave em cada um, o que resulta em aproximadamente $3 \cdot n \cdot \log_2 n$ chamadas, e explica por que, embora seja assintoticamente equivalente ao Merge Sort, a implementação apresentada tem constante maior.

Quanto à memória, todas as implementações copiam a lista de entrada para não alterá-la, o que custa O(n) adicional; o Merge Sort e o Quick Sort criam ainda sublistas em cada nível de recursão. Como a profundidade da recursão é de cerca de $\log_2 n$ (14 níveis para dez mil registros) quando as partições são equilibradas, não há risco de estourar o limite de recursão do Python, cujo padrão é de mil chamadas aninhadas. Esse risco existiria no Quick Sort com pivô ruim em uma lista de dez mil elementos, o que reforça a importância da escolha do pivô central e do tratamento dos elementos iguais.

4.14 Tratamento de erros e validações

O sistema trata os erros previsíveis em três níveis. Primeiro, a validação dos parâmetros: algoritmo, campo e cenário desconhecidos, quantidades não numéricas, latitude e longitude fora de formato e campos obrigatórios ausentes levantam uma exceção própria (Erro), convertida em uma resposta HTTP 400 com mensagem em português. Segundo, a validação das imagens enviadas: o arquivo é aberto com a biblioteca Pillow e rejeitado se não for uma imagem válida, e há limite de 5 MB por envio, acima do qual o servidor responde com erro 413. Terceiro, os erros do banco: violações de integridade e dados inválidos (por exemplo, texto maior que a coluna) são capturados e devolvidos como mensagem ao usuário, e a ausência do MySQL é

detectada pelo script de carga, que orienta o usuário a verificar a conexão. Registros inexistentes resultam em resposta 404.

Também foram previstos o caso do banco vazio (o comparador informa o comando necessário para carregar os dados), a proteção contra a recarga acidental (o script de carga se recusa a rodar se a tabela já tiver registros) e a proteção contra SQL injection nos comandos de atualização, em que os nomes das colunas vêm de uma lista fixa de campos editáveis e os valores sempre são enviados como parâmetros.

4.15 Execução do sistema

Para executar o sistema, são necessários Python 3 e um servidor MySQL (por exemplo, o do XAMPP). Os passos são: instalar as dependências com pip install -r requirements.txt; carregar os dados com python -m core.dados.popular_banco; e iniciar o servidor com python app.py, acessando as páginas em http://localhost:5000. Os parâmetros de conexão com o MySQL têm padrão de instalação local (usuário root, sem senha, porta 3306) e podem ser alterados por variáveis de ambiente. O código-fonte completo está reproduzido na seção 8.

5 RESULTADOS E DISCUSSÃO

5.1 Metodologia dos testes

Os testes foram executados com o script benchmark/medir_desempenho.py, em ambiente Linux com dois processadores virtuais e Python 3.13, com o código exato dos quatro algoritmos apresentados na seção 4. O tempo é o da chamada a sort(), medido com time.perf_counter, sem contar a preparação dos dados. Como a quantidade de elementos define o custo e o campo de ordenação é apenas a chave de comparação, as entradas são registros com os campos id e tamanho_bytes, este com valores inteiros aleatórios entre 5.000 e 400.000, faixa compatível com o tamanho de miniaturas PNG.

Foram testados seis tamanhos de entrada (100; 500; 1.000; 2.500; 5.000; 10.000 registros) em quatro cenários: aleatório (amostra sem ordem); semiordenado (lista ordenada na qual 5% das posições foram trocadas aleatoriamente); ordenado (já em ordem crescente) e invertido (em ordem decrescente). Para cada combinação, os quatro algoritmos receberam exatamente as mesmas listas. Cada medição foi repetida cinco vezes até 1.000 registros, três vezes até 5.000 e duas vezes com 10.000, e adotou-se a mediana dos tempos para reduzir o efeito de oscilações do sistema. Após cada ordenação, o script confere se o resultado é igual ao obtido pela função sorted do Python, o que valida a correção das implementações.

Como a linguagem é interpretada e a comparação é feita por uma função de extração de chave (key), os tempos absolutos são maiores do que seriam em uma linguagem compilada; porém, a relação entre os algoritmos e a forma como cada tempo cresce com n dependem da estrutura de cada algoritmo e permitem a análise pretendida. Os valores das tabelas a seguir estão em milissegundos (ms).

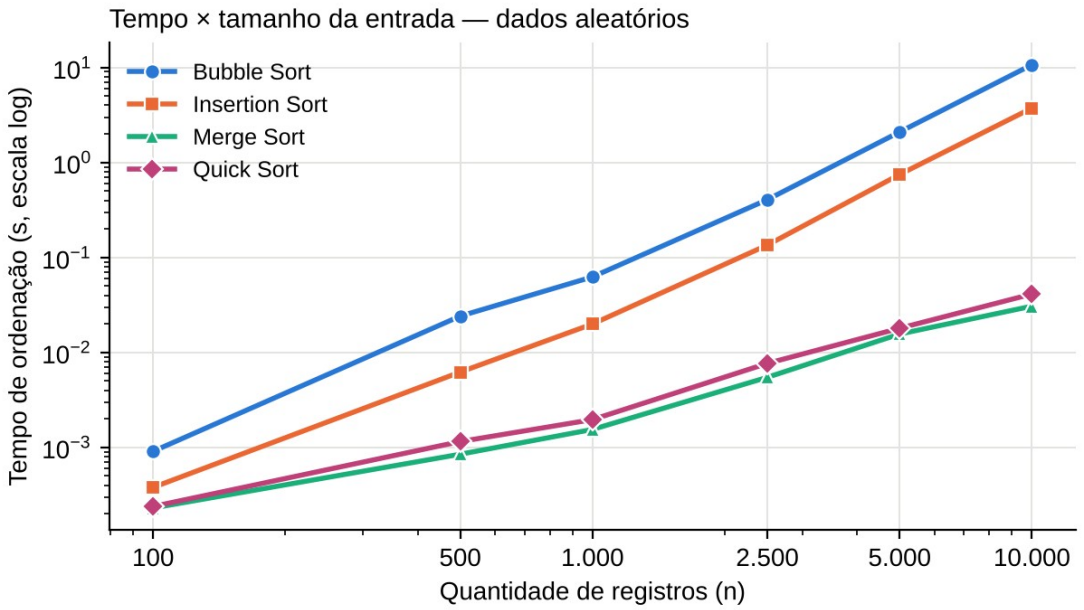
5.2 Dados aleatórios

Tabela 2 – Tempo de ordenação (ms) com dados aleatórios

n	Bubble Sort	Insertion Sort	Merge Sort	Quick Sort
100	0,909	0,379	0,230	0,239
500	24,12	6,21	0,847	1,15
1.000	62,42	19,91	1,55	1,96
2.500	406,20	135,00	5,47	7,67
5.000	2.094,53	747,34	15,62	17,96
10.000	10.680,89	3.735,26	30,68	41,17

Fonte: elaborada pelos autores (medições do script medir_desempenho.py).

Figura 2 – Tempo de ordenação com dados aleatórios



Fonte: elaborada pelos autores. Eixos em escala logarítmica.

Com dados aleatórios, a diferença entre as classes de complexidade é nítida. Com 10.000 registros, o Bubble Sort levou 10.680,89 ms, o Insertion Sort 3.735,26 ms, o Merge Sort 30,68 ms e o Quick Sort 41,17 ms. O Merge Sort foi cerca de 348 vezes mais rápido que o Bubble Sort e 122 vezes mais rápido que o Insertion Sort. O Insertion Sort foi 2,9 vezes mais rápido que o Bubble Sort, coerente com o fato de ambos realizarem número de comparações da mesma ordem, mas o Insertion Sort fazer deslocamentos simples em vez de trocas completas.

O crescimento com n confirma a teoria. Ao dobrar a entrada de 5.000 para 10.000, o tempo do Bubble Sort foi multiplicado por 5,1 e o do Insertion Sort por 5,0,

valores próximos de 4, como esperado para custo quadrático; já os tempos do Merge Sort e do Quick Sort foram multiplicados por 2,0 e 2,3, próximos de 2, como esperado para n log n. Entre os dois algoritmos de divisão e conquista, o Merge Sort foi ligeiramente mais rápido que o Quick Sort em todos os tamanhos. Isso difere da expectativa usual, em que o Quick Sort in-place vence o Merge Sort, e se explica pela implementação: a versão com três listas percorre os dados três vezes por nível de recursão e chama a função de chave em cada percurso, enquanto o Merge Sort faz uma única passada de intercalação.

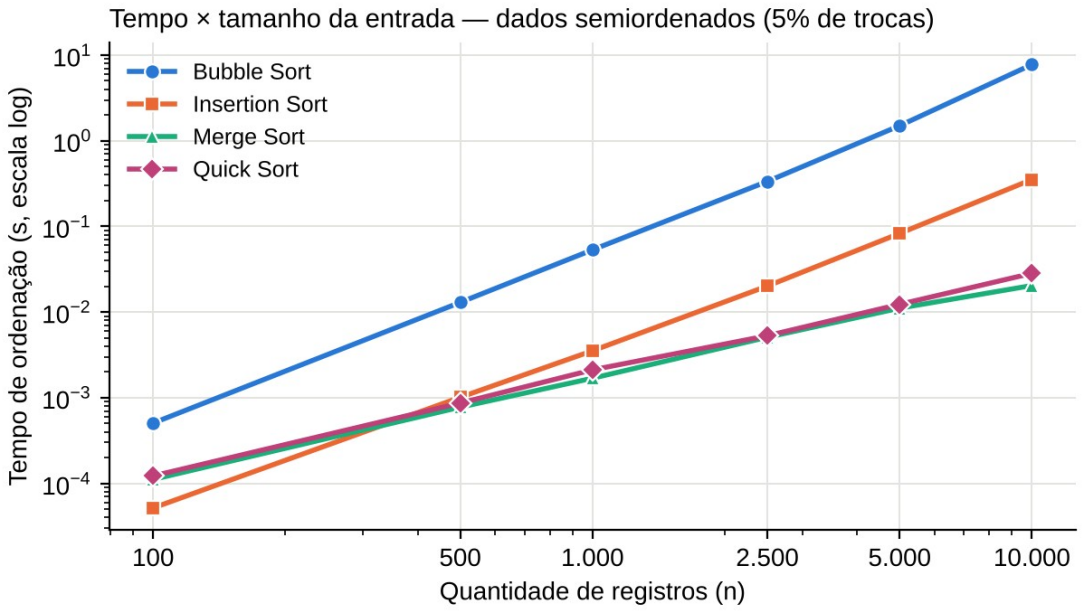
5.3 Dados semiordenados

Tabela 3 – Tempo de ordenação (ms) com dados semiordenados (5% de trocas)

n	Bubble Sort	Insertion Sort	Merge Sort	Quick Sort
100	0,506	0,052	0,111	0,122
500	12,99	1,01	0,777	0,868
1.000	53,34	3,54	1,69	2,12
2.500	335,54	20,14	5,09	5,30
5.000	1.499,20	82,90	11,13	12,33
10.000	7.757,01	353,10	20,43	28,34

Fonte: elaborada pelos autores (medições do script medir_desempenho.py).

Figura 3 – Tempo de ordenação com dados semiordenados



Fonte: elaborada pelos autores. Eixos em escala logarítmica.

Nos dados semiordenados aparece a principal vantagem do Insertion Sort. Com 10.000 registros, ele gastou 353,10 ms, contra 3.735,26 ms nos dados aleatórios, ou seja, 10,6 vezes menos, pois seu custo é proporcional ao número de inversões, que é pequeno quando poucos elementos estão fora do lugar. Ele foi 22,0 vezes mais rápido que o Bubble Sort, que melhora pouco (de 10.680,89 ms para 7.757,01 ms), já que a interrupção antecipada só ajuda quando a lista fica ordenada logo nas primeiras passadas e basta um elemento pequeno estar no fim para exigir muitas passadas. Merge Sort e Quick Sort praticamente não mudaram, o que confirma a insensibilidade desses algoritmos à organização inicial. Vale observar que o ganho do Insertion Sort é de constante, e não de classe de complexidade: como 5% dos elementos estão fora de posição e cada um pode estar distante de seu lugar, o número de inversões ainda cresce com n^2, comportamento que a seção 5.7 quantifica.

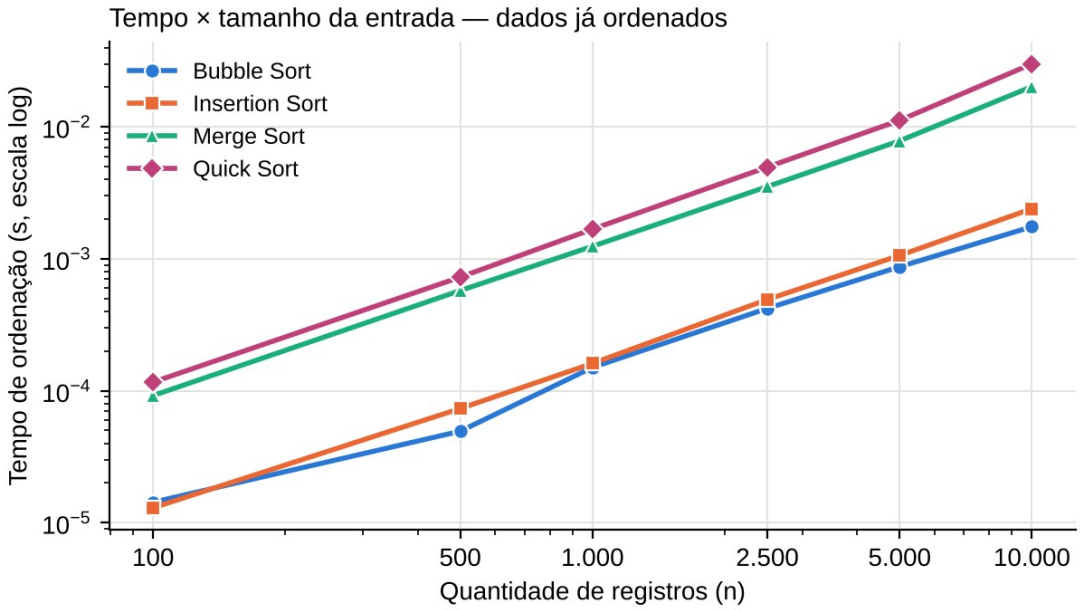
5.4 Dados ordenados

Tabela 4 – Tempo de ordenação (ms) com dados já ordenados

n	Bubble Sort	Insertion Sort	Merge Sort	Quick Sort
100	0,014	0,013	0,092	0,116
500	0,049	0,073	0,575	0,726
1.000	0,150	0,161	1,24	1,68
2.500	0,421	0,492	3,53	4,96
5.000	0,871	1,06	7,84	11,22
10.000	1,75	2,40	20,23	30,06

Fonte: elaborada pelos autores (medições do script medir_desempenho.py).

Figura 4 – Tempo de ordenação com dados já ordenados



Fonte: elaborada pelos autores. Eixos em escala logarítmica.

Este é o cenário em que os algoritmos quadráticos vencem os de divisão e conquista. Com 10.000 registros já ordenados, o Bubble Sort gastou 1,75 ms e o Insertion Sort 2,40 ms, enquanto o Merge Sort gastou 20,23 ms e o Quick Sort 30,06 ms. Isso acontece porque o Bubble Sort realiza uma única passada e termina, e o Insertion Sort apenas confirma, para cada elemento, que ele já está no lugar, ambos com custo linear; já o Merge Sort e o Quick Sort executam todos os níveis de recursão mesmo que nada precise mudar de posição. O resultado reforça que a notação $O(n^2)$ descreve o comportamento no pior caso, e que o melhor algoritmo depende do cenário. Vale notar que o Quick Sort com pivô no elemento central não degenerou: o tempo com dados ordenados é semelhante ao dos dados aleatórios, o que mostra a eficácia da escolha do pivô.

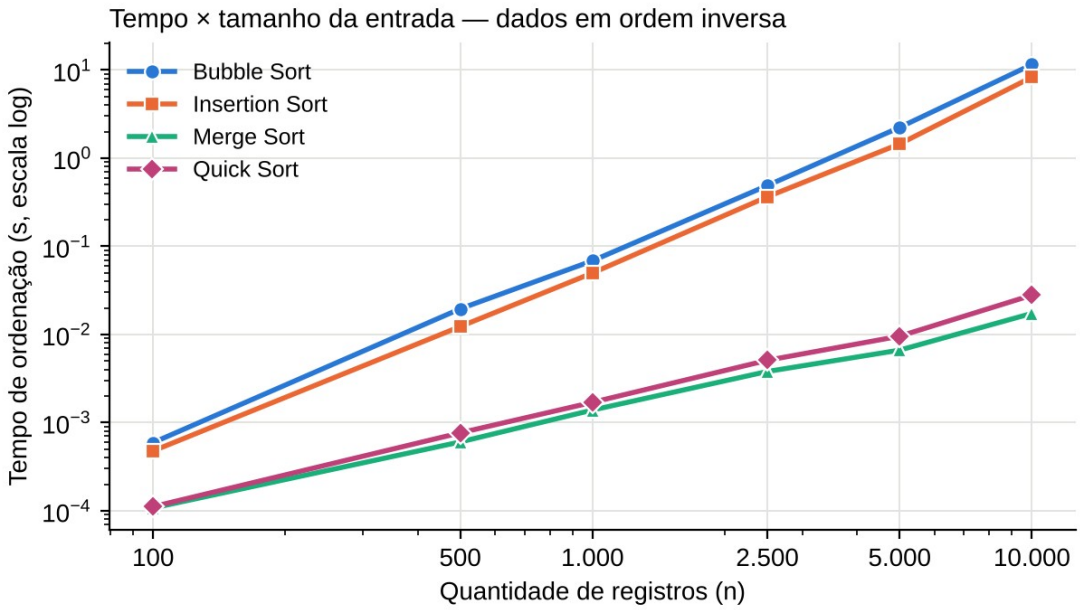
5.5 Dados em ordem inversa

Tabela 5 – Tempo de ordenação (ms) com dados em ordem inversa

n	Bubble Sort	Insertion Sort	Merge Sort	Quick Sort
100	0,592	0,477	0,107	0,111
500	19,54	12,29	0,603	0,765
1.000	68,55	49,82	1,39	1,70
2.500	489,71	364,77	3,80	5,09
5.000	2.235,28	1.451,24	6,64	9,55
10.000	11.543,70	8.299,57	17,33	27,90

Fonte: elaborada pelos autores (medições do script medir_desempenho.py).

Figura 5 – Tempo de ordenação com dados em ordem inversa

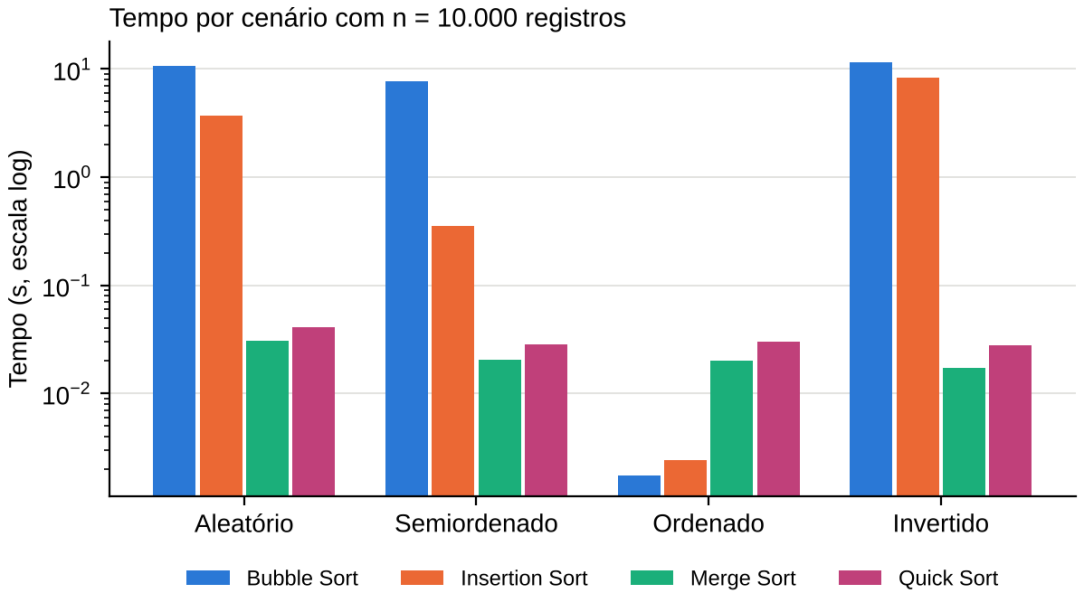


Fonte: elaborada pelos autores. Eixos em escala logarítmica.

Em ordem inversa, todas as comparações resultam em troca ou deslocamento, o que caracteriza o pior caso dos algoritmos quadráticos. Com 10.000 registros, o Bubble Sort levou 11.543,70 ms e o Insertion Sort 8.299,57 ms, o maior valor da série do Bubble Sort, enquanto Merge Sort e Quick Sort levaram 17,33 ms e 27,90 ms. Curiosamente, os algoritmos de divisão e conquista foram até mais rápidos que nos dados aleatórios, pois a lista invertida produz partições e intercalações regulares, que favorecem a previsão de desvios do processador. O Insertion Sort, que nos dados aleatórios realiza em média metade do deslocamento, aqui realiza o deslocamento máximo, e por isso seu tempo se aproxima do tempo do Bubble Sort.

5.6 Síntese e discussão

Figura 6 – Tempo por cenário com n = 10.000 registros



Fonte: elaborada pelos autores. Eixo vertical em escala logarítmica.

Tabela 6 – Vantagens e desvantagens observadas em cada algoritmo

Algoritmo	Vantagens observadas	Desvantagens observadas
Bubble Sort	Muito simples; linear com dados já ordenados.	Maior tempo na maioria dos cenários; quadrático em dados aleatórios e invertidos.
Insertion Sort	Melhor com dados ordenados ou quase ordenados; superior ao Bubble Sort.	Quadrático em dados aleatórios e invertidos.
Merge Sort	Tempo previsível em todos os cenários; mais rápido com dados aleatórios.	Mais lento que os quadráticos com dados ordenados; usa memória auxiliar.
Quick Sort	Escalável e rápido em todos os cenários com o pivô central.	Pior caso teórico quadrático; versão com listas usa mais memória e passadas.

Fonte: elaborada pelos autores.

A Figura 6 e a Tabela 6 resumem a comparação. Dois padrões se destacam. O primeiro é a separação por ordens de grandeza entre os algoritmos quadráticos e os de divisão e conquista nos cenários aleatório e invertido, com diferença de até 666 vezes entre o Bubble Sort e o Merge Sort com 10.000 registros em ordem inversa. O segundo é a dependência dos quadráticos em relação à organização inicial: o Insertion

Sort variou de 2,40 ms (ordenado) a 8.299,57 ms (invertido) com a mesma quantidade de dados, enquanto o Merge Sort variou apenas de 17,33 ms a 30,68 ms.

Para o catálogo de imagens do Cerrado, a discussão traz implicações práticas. O catálogo recebe novas imagens continuamente, e uma lista que já foi ordenada e recebe poucos registros novos fica semiordenada; nesse perfil, o Insertion Sort é uma opção competitiva para manutenção incremental. Para reordenar todo o conjunto por um novo critério, ou para volumes na casa de dezenas ou centenas de milhares de registros, como os citados no enunciado, os algoritmos quadráticos são inviáveis: extrapolando o crescimento quadrático observado, o Bubble Sort levaria cerca de quinze a vinte minutos para ordenar cem mil registros aleatórios nesta mesma implementação, enquanto Merge Sort e Quick Sort levariam algo em torno de meio segundo. Entre os dois, o Merge Sort é preferível quando se deseja previsibilidade e estabilidade, e o Quick Sort, quando se busca desempenho médio e economia de memória em sua versão in-place.

Há limitações que devem ser consideradas ao interpretar os números. Os tempos dependem do computador, do interpretador e da carga do sistema no momento do teste, razão pela qual se adotou a mediana de repetições; a comparação relativa entre os algoritmos é mais confiável que os valores absolutos. Além disso, os testes usaram chaves inteiras geradas aleatoriamente na faixa de tamanhos de miniaturas, e não as chaves de texto (nome, satélite, data), cuja comparação é mais custosa e poderia ampliar as diferenças absolutas sem alterar a ordem entre os algoritmos.

5.7 Expoente empírico de crescimento

Para quantificar o crescimento dos tempos, calculou-se, para cada algoritmo e cenário, o expoente empírico k entre os tamanhos 1.000 e 10.000, supondo que o tempo seja proporcional a n elevado a k. O expoente é dado por $k = \log(t_{10000} / t_{1000}) / \log(10)$. Um algoritmo quadrático deve apresentar k próximo de 2, um linear, próximo de 1, e um de custo n log n, ligeiramente acima de 1 (cerca de 1,1 a 1,3 nessa faixa de tamanhos).

Tabela 7 – Expoente empírico k de crescimento do tempo (n de 1.000 a 10.000)

Algoritmo	Aleatório	Semiordenado	Ordenado	Invertido
Bubble Sort	2,23	2,16	1,07	2,23
Insertion Sort	2,27	2,00	1,17	2,22
Merge Sort	1,30	1,08	1,21	1,10
Quick Sort	1,32	1,13	1,25	1,22

Fonte: elaborada pelos autores, a partir das medições das Tabelas 2 a 5.

Os expoentes confirmam a análise teórica. No cenário aleatório, o Bubble Sort apresentou k = 2,23 e o Insertion Sort k = 2,27, valores próximos de 2, enquanto o Merge Sort apresentou k = 1,30 e o Quick Sort k = 1,32, compatíveis com n log n. Nos dados ordenados, o Bubble Sort (k = 1,07) e o Insertion Sort (k = 1,17) apresentaram expoente próximo de 1, o que caracteriza custo linear, e nos dados semiordenados o Insertion Sort apresentou k = 2,00: embora seja cerca de 10,6 vezes mais rápido que nos dados aleatórios, seu crescimento continua quadrático, pois, mantida a proporção de 5% de elementos fora do lugar, o número de inversões ainda cresce com o quadrado de n. Desvios em relação aos valores ideais são esperados, porque em tamanhos pequenos o custo fixo de cada chamada tem peso relativo maior e porque a coleta de lixo e a memória cache do processador influenciam as execuções.

5.8 Confiabilidade dos resultados

Três cuidados sustentam a confiabilidade das medições. O primeiro é a correção: o script conferiu o resultado de cada ordenação contra a função sorted do Python, e nenhuma divergência foi encontrada. O segundo é a equidade: os quatro algoritmos receberam exatamente as mesmas listas em cada repetição, e todos retornam cópias, sem alterar a entrada. O terceiro é a robustez estatística: usou-se a mediana de várias repetições, pouco sensível a execuções isoladas afetadas por outros processos do sistema. Ainda assim, recomenda-se repetir o experimento em outros computadores, o que é simples: basta executar python -m benchmark.medir_desempenho na raiz do projeto, que regrava o arquivo de resultados.

6 CONSIDERAÇÕES FINAIS

O trabalho atingiu seu objetivo principal: foi construído um sistema completo, em Python, Flask e MySQL, que obtém metadados e miniaturas reais de imagens de satélite do Cerrado Goiano a partir do catálogo do INPE, armazena as imagens em campo BLOB, permite cadastrar, consultar, alterar e excluir registros, ordena o catálogo por diferentes campos com quatro algoritmos e mede com precisão apenas o tempo da ordenação, deixando de fora a aquisição dos dados.

Do ponto de vista dos algoritmos, as medições confirmaram o que a teoria prevê. Bubble Sort e Insertion Sort têm custo quadrático e se tornam impraticáveis a partir de alguns milhares de registros em dados desordenados, mas são imbatíveis quando os dados já estão ordenados, e o Insertion Sort se destaca em dados quase ordenados. Merge Sort e Quick Sort mantiveram tempos baixos e crescimento próximo de n log n em todos os cenários, e o primeiro foi um pouco mais rápido nesta implementação funcional. A principal lição é que não existe um algoritmo melhor em todos os casos: a escolha depende do volume de dados, da organização inicial, da necessidade de estabilidade e da memória disponível.

O desenvolvimento também deixou aprendizados de engenharia de software: a separação entre leitura dos metadados e das imagens para economizar memória; a validação de entradas e o tratamento de erros em todas as camadas; o cuidado de posicionar o cronômetro apenas ao redor da ordenação; a necessidade de adaptar o projeto às exigências do ambiente, como a migração para o MySQL; e a importância de simplificar o escopo para que o grupo domine integralmente o que apresenta.

Em relação aos objetivos específicos, todos foram cumpridos: os dados reais foram obtidos e armazenados com a imagem em BLOB; os quatro algoritmos foram implementados com interface comum; a aplicação web oferece o CRUD completo, a ordenação e o comparador; os testes cobriram seis tamanhos de entrada e quatro cenários, apresentados em tabelas e gráficos; e os resultados foram discutidos à luz da complexidade teórica.

Entre as dificuldades encontradas, destacam-se a necessidade de filtrar as cenas do catálogo do INPE para que pertencessem de fato ao Cerrado Goiano, pois algumas coleções contêm produtos de escala continental; a decisão de usar somente as miniaturas, já que o arquivo completo de cada cena tem cerca de um gigabyte; a migração do banco SQLite para o MySQL, que exigiu adaptar os tipos de dados e o

processo de importação; e o limite de tamanho de arquivo para importação no phpMyAdmin, contornado por um script de importação próprio. Cada uma delas foi resolvida e documentada no código.

Como trabalhos futuros, o grupo sugere incluir outros algoritmos, como o Heap Sort e o Shell Sort, e variantes do Quick Sort in-place e com mediana de três; ampliar os testes para entradas de maior tamanho e para chaves de texto; permitir que o comparador gere gráficos na própria interface; e explorar a ordenação externa por intercalação, adequada ao volume diário de imagens citado no enunciado, em que os dados não cabem inteiros na memória.

7 REFERÊNCIAS BIBLIOGRÁFICAS

ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS. **NBR 6023**: informação e documentação: referências: elaboração. Rio de Janeiro: ABNT, 2018.

ASTRACHAN, O. Bubble sort: an archaeological algorithmic analysis. **ACM SIGCSE Bulletin**, v. 35, n. 1, p. 1-5, 2003.

BRAZIL DATA CUBE. **STAC Brazil Data Cube**. São José dos Campos: INPE, 2026. Disponível em: https://data.inpe.br/bdc/stac/v1. Acesso em: 28 out. 2026.

CORMEN, T. H.; LEISERSON, C. E.; RIVEST, R. L.; STEIN, C. **Algoritmos**: teoria e prática. 3. ed. Rio de Janeiro: Elsevier, 2012.

ELMASRI, R.; NAVATHE, S. B. **Sistemas de banco de dados**. 7. ed. São Paulo: Pearson, 2019.

GRINBERG, M. **Flask web development**: developing web applications with Python. 2. ed. Sebastopol: O'Reilly, 2018.

HOARE, C. A. R. Quicksort. **The Computer Journal**, v. 5, n. 1, p. 10-16, 1962.

INSTITUTO NACIONAL DE PESQUISAS ESPACIAIS. **PRODES Cerrado**: monitoramento do desmatamento no bioma Cerrado por satélite. São José dos Campos: INPE, 2026. Disponível em: https://terrabrasilis.dpi.inpe.br. Acesso em: 28 out. 2026.

KNUTH, D. E. **The art of computer programming**: sorting and searching. 2. ed. Reading: Addison-Wesley, 1998. v. 3.

ORACLE. **MySQL 8.0 reference manual**: the BLOB and TEXT types. Redwood Shores: Oracle, 2026. Disponível em: https://dev.mysql.com/doc/refman/8.0/en/blob.html. Acesso em: 28 out. 2026.

PALLETS PROJECTS. **Flask documentation**. 2026. Disponível em: https://flask.palletsprojects.com. Acesso em: 28 out. 2026.

PYMYSQL. **PyMySQL documentation**. 2026. Disponível em: https://pymysql.readthedocs.io. Acesso em: 28 out. 2026.

PYTHON SOFTWARE FOUNDATION. **Python 3 documentation**: time.perf_counter; sorting HOW TO. 2026. Disponível em: https://docs.python.org/3. Acesso em: 28 out. 2026.

SEDGEWICK, R. Implementing quicksort programs. **Communications of the ACM**, v. 21, n. 10, p. 847-857, 1978.

SEDGEWICK, R.; WAYNE, K. **Algorithms**. 4. ed. Upper Saddle River: Addison-Wesley, 2011.

SHELL, D. L. A high-speed sorting procedure. **Communications of the ACM**, v. 2, n. 7, p. 30-32, 1959.

SKIENA, S. S. **The algorithm design manual**. 2. ed. London: Springer, 2008.

SPATIOTEMPORAL ASSET CATALOG. **STAC specification**. 2026. Disponível em: https://stacspec.org. Acesso em: 28 out. 2026.

SZWARCFITER, J. L.; MARKENZON, L. **Estruturas de dados e seus algoritmos**. 3. ed. Rio de Janeiro: LTC, 2010.

WILLIAMS, J. W. J. Algorithm 232: heapsort. **Communications of the ACM**, v. 7, n. 6, p. 347-348, 1964.

ZIVIANI, N. **Projeto de algoritmos**: com implementações em Java e C++. São Paulo: Thomson Learning, 2007.

8 CÓDIGO FONTE

Esta seção reproduz o código-fonte completo do sistema, na ordem: servidor (app.py), acesso a dados (core/dados), algoritmos (core/algoritmos), script de medição (benchmark), interface (static) e dependências. Para legibilidade, os códigos usam fonte monoespaçada em corpo menor e espaçamento simples.

8.1 app.py

```
"""
app.py — Servidor Flask: entrega as 3 páginas HTML e liga os algoritmos aos dados do banco.

Rodar:  python app.py   ->  http://localhost:5000
"""

import os
import random
import time

import pymysql
from flask import Flask, Response, jsonify, redirect, request, send_from_directory

from core.algoritmos import bubble_sort, insertion_sort, merge_sort, quick_sort
from core.dados import db
from core.dados.imagens import detectar_mime

app = Flask(__name__)
app.config["BANCO"] = db.NOME_BANCO
app.config["MAX_CONTENT_LENGTH"] = 5 * 1024 * 1024  # imagens enviadas no CRUD: até 5 MB

ALGORITMOS = {
    "bubble": ("Bubble Sort", bubble_sort),
    "insertion": ("Insertion Sort", insertion_sort),
    "merge": ("Merge Sort", merge_sort),
    "quick": ("Quick Sort", quick_sort),
}
CAMPOS = ("tamanho_bytes", "data_captura", "nome_arquivo", "satelite", "id")


class Erro(Exception):
    """Pedido inválido: vira uma resposta 400 com a mensagem."""


@app.errorhandler(Erro)
def _erro(e):
    return jsonify({"erro": str(e)}), 400


@app.errorhandler(413)
def _arquivo_grande(e):
    return jsonify({"erro": "a imagem passa de 5 MB"}), 413


def _banco():
    return db.conectar(app.config["BANCO"])


def _chave(campo):
    """Como ler o valor que será ordenado. Valor vazio (None) vira '' para não quebrar a
comparação."""
    if campo in ("tamanho_bytes", "id"):
        return lambda r: r[campo]
    return lambda r: r[campo] or ""


def _validar_ordenacao(algoritmo, campo):
    if algoritmo not in ALGORITMOS:
        raise Erro(f"algoritmo inválido: {algoritmo}")
    if campo not in CAMPOS:
```

```
        raise Erro(f"campo inválido: {campo}")


# ------------------------------------------------------------------------
# Páginas
# ------------------------------------------------------------------------

@app.route("/")
def inicio():
    return redirect("/ordenacao")


@app.route("/ordenacao")
def pagina_ordenacao():
    return send_from_directory("static", "ordenacao.html")


@app.route("/crud")
def pagina_crud():
    return send_from_directory("static", "crud.html")


@app.route("/comparador")
def pagina_comparador():
    return send_from_directory("static", "comparador.html")


# ------------------------------------------------------------------------
# Página 1: ordenação com um algoritmo
# ------------------------------------------------------------------------

@app.route("/api/ordenar")
def ordenar():
    algoritmo = request.args.get("algoritmo", "quick")
    campo = request.args.get("campo", "tamanho_bytes")
    _validar_ordenacao(algoritmo, campo)

    with _banco() as conn:
        registros = db.listar_todos(conn)

    inicio = time.perf_counter()  # o cronômetro mede SÓ o sort(), sem a leitura do banco
    ordenados = ALGORITMOS[algoritmo][1].sort(registros, key=_chave(campo))
    tempo = time.perf_counter() - inicio

    return jsonify({
        "algoritmo": ALGORITMOS[algoritmo][0],
        "campo": campo,
        "quantidade": len(ordenados),
        "tempo_segundos": tempo,
        "itens": ordenados,
    })


# ------------------------------------------------------------------------
# Página 3: comparador
# ------------------------------------------------------------------------

@app.route("/api/resumo")
def resumo():
    with _banco() as conn:
        return jsonify({"total": db.contar(conn)})


@app.route("/api/comparar")
def comparar():
    campo = request.args.get("campo", "tamanho_bytes")
    cenario = request.args.get("cenario", "aleatorio")
    try:
        quantidade = int(request.args.get("quantidade", 1000))
    except ValueError:
        raise Erro("quantidade precisa ser um número inteiro")
    if campo not in CAMPOS:
        raise Erro(f"campo inválido: {campo}")
    if cenario not in ("aleatorio", "ordenado", "invertido"):
        raise Erro(f"cenário inválido: {cenario}")

    with _banco() as conn:
```

```
        registros = db.listar_todos(conn)
    if not registros:
        raise Erro("o banco está vazio. Rode: python -m core.dados.popular_banco")

    quantidade = max(1, min(quantidade, len(registros)))
    amostra = random.sample(registros, quantidade)  # amostra aleatória do banco
    chave = _chave(campo)
    if cenario == "ordenado":
        amostra.sort(key=chave)               # prepara os dados ANTES de cronometrar
    elif cenario == "invertido":
        amostra.sort(key=chave, reverse=True)

    resultados = []
    for nome, (rotulo, modulo) in ALGORITMOS.items():
        inicio = time.perf_counter()
        modulo.sort(amostra, key=chave)
        resultados.append({"algoritmo": rotulo, "tempo_segundos": time.perf_counter() -
inicio})

    return jsonify({"quantidade": quantidade, "campo": campo, "cenario": cenario,
"resultados": resultados})


# ---------------------------------------------------------------------------
# Página 2: CRUD
# ---------------------------------------------------------------------------

def _numero(valor, nome):
    if valor in (None, ""):
        return None
    try:
        return float(valor)
    except (TypeError, ValueError):
        raise Erro(f"{nome} precisa ser um número")


def _campos_do_pedido(dados, obrigatorio):
    """Lê e confere os metadados enviados pelo formulário (dados = request.form ou JSON)."""
    campos = {}
    for nome in db.CAMPOS_EDITAVEIS:
        if nome in dados:
            valor = dados[nome]
            campos[nome] = _numero(valor, nome) if nome in ("latitude", "longitude") else
(valor or None)
    if obrigatorio and not campos.get("nome_arquivo"):
        raise Erro("nome_arquivo é obrigatório")
    if "nome_arquivo" in campos and not campos["nome_arquivo"]:
        raise Erro("nome_arquivo não pode ficar vazio")
    return campos


@app.route("/api/registros", methods=["GET"])
def listar_registros():
    id_busca = request.args.get("id", "").strip()
    if id_busca:  # busca por ID: devolve só esse registro (ou lista vazia)
        try:
            id_ = int(id_busca)
        except ValueError:
            raise Erro("o ID precisa ser um número inteiro")
        with _banco() as conn:
            registro = db.obter(conn, id_)
        itens = [registro] if registro else []
        return jsonify({"itens": itens, "total": len(itens), "pagina": 1, "total_paginas": 1})
    try:
        pagina = max(1, int(request.args.get("pagina", 1)))
    except ValueError:
        raise Erro("pagina precisa ser um número inteiro")
    por_pagina = 20
    with _banco() as conn:
        itens, total = db.listar_pagina(conn, pagina, por_pagina)
    return jsonify({"itens": itens, "total": total, "pagina": pagina,
                    "total_paginas": max(1, -(-total // por_pagina))})


@app.route("/api/registros", methods=["POST"])
def criar_registro():
    campos = _campos_do_pedido(request.form, obrigatorio=True)
```

```
    arquivo = request.files.get("imagem")
    if not arquivo:
        raise Erro("envie uma imagem")
    imagem = arquivo.read()
    if detectar_mime(imagem) is None:
        raise Erro("o arquivo enviado não é uma imagem válida (PNG, JPEG, GIF ou WEBP)")
    try:
        with _banco() as conn:
            novo_id = db.criar(conn, campos, imagem)
            return jsonify(db.obter(conn, novo_id)), 201
    except (pymysql.err.IntegrityError, pymysql.err.DataError) as erro:
        raise Erro(f"não foi possível gravar: {erro}")


@app.route("/api/registros/<int:id_>", methods=["PUT"])
def editar_registro(id_):
    campos = _campos_do_pedido(request.form, obrigatorio=False)
    arquivo = request.files.get("imagem")
    imagem = arquivo.read() if arquivo and arquivo.filename else None  # imagem nova é
opcional
    if imagem is not None and detectar_mime(imagem) is None:
        raise Erro("o arquivo enviado não é uma imagem válida (PNG, JPEG, GIF ou WEBP)")
    with _banco() as conn:
        if not db.atualizar(conn, id_, campos, imagem):
            return jsonify({"erro": "registro não encontrado"}), 404
        return jsonify(db.obter(conn, id_))


@app.route("/api/registros/<int:id_>", methods=["DELETE"])
def apagar_registro(id_):
    with _banco() as conn:
        if not db.deletar(conn, id_):
            return jsonify({"erro": "registro não encontrado"}), 404
    return jsonify({"apagado": id_})


@app.route("/api/imagens/<int:id_>/thumbnail")
def thumbnail(id_):
    with _banco() as conn:
        blob = db.obter_thumbnail(conn, id_)
    mime = detectar_mime(blob) if blob else None
    if mime is None:
        return jsonify({"erro": "sem imagem"}), 404
    resposta = Response(blob, mimetype=mime)
    resposta.headers["Cache-Control"] = "no-cache"  # depois de trocar a imagem, o navegador
busca a nova
    return resposta


if __name__ == "__main__":
    app.run(host="0.0.0.0", port=int(os.environ.get("PORT", 5000)))
```

8.2 core/dados/db.py

```
"""
db.py — Conexão com o MySQL e as operações da tabela (inserir, listar, editar, apagar).

Uma tabela só: metadados_imagem. O thumbnail fica na coluna thumbnail_blob (BLOB = os
bytes da imagem guardados dentro do próprio banco).

Os dados de conexão têm padrão de XAMPP/WAMP (usuário root, sem senha, porta 3306).
Para mudar, defina variáveis de ambiente antes de rodar:
    MYSQL_HOST, MYSQL_PORT, MYSQL_USER, MYSQL_PASSWORD, MYSQL_DATABASE

O banco (database) é criado sozinho se não existir. Dá para ver as tabelas e linhas
pelo phpMyAdmin: http://localhost/phpmyadmin
"""

import os
from contextlib import contextmanager

import pymysql
from pymysql.cursors import DictCursor

CONFIG = {
```

```
    "host": os.environ.get("MYSQL_HOST", "localhost"),
    "port": int(os.environ.get("MYSQL_PORT", "3306")),
    "user": os.environ.get("MYSQL_USER", "root"),
    "password": os.environ.get("MYSQL_PASSWORD", ""),
}
NOME_BANCO = os.environ.get("MYSQL_DATABASE", "catalogo_cerrado")

# CREATE TABLE IF NOT EXISTS: cria a tabela só se ela ainda não existir.
#   INT AUTO_INCREMENT PRIMARY KEY  -> id numérico que o próprio MySQL numera (1, 2, 3...)
#   VARCHAR(n)                      -> texto de até n caracteres
#   DOUBLE                          -> número com casas decimais (latitude/longitude)
#   MEDIUMBLOB                      -> bytes de até 16 MB (um BLOB comum guarda só 64 KB)
#   ENUM(...)                       -> a coluna só aceita um dos valores da lista. É o que diz
#                                      de onde veio a linha: real (INPE) ou manual (cadastrada
pelo CRUD)
#   TIMESTAMP DEFAULT CURRENT_TIMESTAMP -> data/hora preenchida sozinha no INSERT
#   utf8mb4                         -> codificação que aceita acentos e qualquer caractere
SCHEMA = """
CREATE TABLE IF NOT EXISTS metadados_imagem (
    id              INT AUTO_INCREMENT PRIMARY KEY,
    nome_arquivo    VARCHAR(255) NOT NULL,
    data_captura    VARCHAR(40),
    satelite        VARCHAR(100),
    sensor          VARCHAR(100),
    latitude        DOUBLE,
    longitude       DOUBLE,
    tamanho_bytes   INT NOT NULL,
    thumbnail_blob  MEDIUMBLOB NOT NULL,
    origem          ENUM('real', 'manual') NOT NULL,
    criado_em       TIMESTAMP DEFAULT CURRENT_TIMESTAMP
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4
"""

CAMPOS_EDITAVEIS = ("nome_arquivo", "data_captura", "satelite", "sensor", "latitude",
"longitude")

# Todas as colunas menos o BLOB: listar milhares de linhas sem carregar as imagens é bem mais
rápido.
# CAST(criado_em AS CHAR) transforma a data em texto ('2026-10-25 15:07:18'), que o JSON
entende.
COLUNAS = ("id, nome_arquivo, data_captura, satelite, sensor, latitude, longitude, "
           "tamanho_bytes, origem, CAST(criado_em AS CHAR) AS criado_em")


@contextmanager
def conectar(banco=NOME_BANCO):
    """Abre o banco, salva (commit) no final e fecha. Uso: with conectar() as conn: ..."""
    conn = pymysql.connect(**CONFIG, charset="utf8mb4", cursorclass=DictCursor)
    try:
        with conn.cursor() as cur:
            # Crases (`) protegem o nome do banco. USE escolhe em qual banco os comandos vão
rodar.
            cur.execute(f"CREATE DATABASE IF NOT EXISTS `{banco}` CHARACTER SET utf8mb4")
            cur.execute(f"USE `{banco}`")
            cur.execute(SCHEMA)
        yield conn
        conn.commit()
    except Exception:
        conn.rollback()
        raise
    finally:
        conn.close()


def _executar(conn, sql, params=()):
    """Roda um comando. Os %s do SQL são preenchidos com 'params' (evita SQL injection)."""
    cur = conn.cursor()
    cur.execute(sql, params)
    return cur


def inserir(conn, registro: dict) -> int:
    """INSERT INTO: adiciona uma linha. Devolve o id que o MySQL deu a ela."""
    cur = _executar(
        conn,
        """INSERT INTO metadados_imagem
           (nome_arquivo, data_captura, satelite, sensor, latitude, longitude,
```

```
            tamanho_bytes, thumbnail_blob, origem)
           VALUES (%s, %s, %s, %s, %s, %s, %s, %s, %s)""",
        (registro["nome_arquivo"], registro.get("data_captura"), registro.get("satelite"),
         registro.get("sensor"), registro.get("latitude"), registro.get("longitude"),
         registro["tamanho_bytes"], registro["thumbnail_blob"], registro["origem"]),
    )
    return cur.lastrowid


def contar(conn) -> int:
    """SELECT COUNT(*): quantas linhas existem na tabela."""
    cur = _executar(conn, "SELECT COUNT(*) AS total FROM metadados_imagem")
    return cur.fetchone()["total"]


def listar_todos(conn) -> list[dict]:
    """SELECT: todas as linhas (sem o BLOB), como lista de dicionários. É isto que vai para os
algoritmos."""
    cur = _executar(conn, f"SELECT {COLUNAS} FROM metadados_imagem")
    return cur.fetchall()


def listar_pagina(conn, pagina: int, por_pagina: int):
    """Uma página de linhas, as mais novas primeiro. LIMIT = quantas; OFFSET = quantas
pular."""
    cur = _executar(
        conn,
        f"SELECT {COLUNAS} FROM metadados_imagem ORDER BY id DESC LIMIT %s OFFSET %s",
        (por_pagina, (pagina - 1) * por_pagina),
    )
    return cur.fetchall(), contar(conn)


def obter(conn, id_: int):
    cur = _executar(conn, f"SELECT {COLUNAS} FROM metadados_imagem WHERE id = %s", (id_,))
    return cur.fetchone()  # None se não existir


def obter_thumbnail(conn, id_: int):
    cur = _executar(conn, "SELECT thumbnail_blob FROM metadados_imagem WHERE id = %s", (id_,))
    linha = cur.fetchone()
    return linha["thumbnail_blob"] if linha else None


def criar(conn, campos: dict, imagem: bytes) -> int:
    """Cadastro pelo CRUD: o tamanho vem da própria imagem e a origem é 'manual'."""
    return inserir(conn, {**campos, "tamanho_bytes": len(imagem), "thumbnail_blob": imagem,
"origem": "manual"})


def atualizar(conn, id_: int, campos: dict, imagem: bytes | None = None) -> bool:
    """
    UPDATE ... SET ... WHERE id = %s: muda só os campos informados. False se o id não existe.
    Se vier uma imagem nova, troca também o thumbnail_blob e recalcula o tamanho_bytes.
    """
    campos = {k: v for k, v in campos.items() if k in CAMPOS_EDITAVEIS}
    if imagem is not None:
        campos["thumbnail_blob"] = imagem
        campos["tamanho_bytes"] = len(imagem)
    if campos:
        trecho = ", ".join(f"{k} = %s" for k in campos)  # nomes vêm de lista fixa, nunca do
usuário
        _executar(conn, f"UPDATE metadados_imagem SET {trecho} WHERE id = %s",
(*campos.values(), id_))
    return obter(conn, id_) is not None


def deletar(conn, id_: int) -> bool:
    """DELETE FROM ... WHERE id = %s: apaga a linha. False se o id não existia."""
    return _executar(conn, "DELETE FROM metadados_imagem WHERE id = %s", (id_,)).rowcount > 0
```

8.3 core/dados/api_inpe.py

```
"""
```

```
api_inpe.py — Busca cenas do Cerrado Goiano no catálogo STAC do INPE e baixa os thumbnails
(PNG).

Fluxo: buscar_itens() -> extrair_metadados() + baixar_thumbnail()
"""

import requests

from core.dados.imagens import detectar_mime

STAC_BASE_URL = "https://data.inpe.br/bdc/stac/v1"
BBOX_CERRADO_GO = (-50.5, -19.0, -46.0, -13.0)  # (lon_min, lat_min, lon_max, lat_max)
HEADERS = {"User-Agent": "APS-UniversidadeGrupo/1.0 (uso academico)"}

# id da coleção -> (satélite, sensor). Só coleções que têm thumbnail PNG.
COLECOES_CERRADO = {
    "S2_L1C_BUNDLE-1": ("SENTINEL-2", "MSI"),
    "CB4-MUX-L4-DN-1": ("CBERS-4", "MUX"),
    "AMZ1-WFI-L2-DN-1": ("AMAZONIA-1", "WFI"),
}
MAX_LADO_CENA_GRAUS = 12.0

# Sessão HTTP: reaproveita a conexão entre os downloads (muito mais rápido
# do que abrir uma conexão nova para cada thumbnail).
SESSAO = requests.Session()
SESSAO.headers.update(HEADERS)


def cena_pertence_ao_cerrado(item, bbox=BBOX_CERRADO_GO, max_lado=MAX_LADO_CENA_GRAUS) ->
bool:
    """Funil: descarta cena gigante (produto global) e cena com o centro fora do Cerrado."""
    caixa = item.get("bbox")
    if not caixa or len(caixa) != 4:
        return False
    min_lon, min_lat, max_lon, max_lat = caixa
    if (max_lon - min_lon) > max_lado or (max_lat - min_lat) > max_lado:
        return False
    lon, lat = (min_lon + max_lon) / 2, (min_lat + max_lat) / 2
    return bbox[0] <= lon <= bbox[2] and bbox[1] <= lat <= bbox[3]


def _buscar_colecao(colecao, bbox, data_inicio, data_fim, maximo):
    """Busca UMA coleção, seguindo o link 'next' de cada página."""
    url = f"{STAC_BASE_URL}/search"
    params = {
        "collections": colecao,  # sem isso vêm cenas de qualquer coleção (radar, GOES...)
        "bbox": ",".join(str(c) for c in bbox),
        "datetime": f"{data_inicio}T00:00:00Z/{data_fim}T23:59:59Z",
        "limit": 500,
    }
    aceitos, vistos = [], set()
    while url and len(aceitos) < maximo:
        resp = SESSAO.get(url, params=params, timeout=30)
        resp.raise_for_status()
        corpo = resp.json()
        features = corpo.get("features", [])
        if not features:
            break
        for item in features:
            if item.get("id") not in vistos and cena_pertence_ao_cerrado(item, bbox):
                vistos.add(item.get("id"))
                aceitos.append(item)
        url = next((l["href"] for l in corpo.get("links", []) if l.get("rel") == "next"),
None)
        params = None  # o link 'next' já traz os filtros
    return aceitos[:maximo]


def buscar_itens(bbox=BBOX_CERRADO_GO, data_inicio="2020-01-01", data_fim="2024-12-31",
                 collections=None, maximo_itens=5000):
    """Cenas do Cerrado. Busca coleção por coleção e só passa para a próxima se faltar
cena."""
    itens = []
    for colecao in (collections or COLECOES_CERRADO):
        faltam = maximo_itens - len(itens)
        itens.extend(_buscar_colecao(colecao, bbox, data_inicio, data_fim, faltam))
        if len(itens) >= maximo_itens:
```

```
            break
    return itens[:maximo_itens]


def extrair_metadados(item) -> dict:
    """Transforma o JSON da cena nos campos da nossa tabela."""
    props = item.get("properties", {})
    sat_padrao, sensor_padrao = COLECOES_CERRADO.get(item.get("collection", ""), (None, None))
    min_lon, min_lat, max_lon, max_lat = item["bbox"]
    instrumentos = props.get("instruments") or []
    return {
        "nome_arquivo": item.get("id", "desconhecido"),
        "data_captura": props.get("datetime") or props.get("start_datetime"),
        "satelite": props.get("platform") or props.get("platformShortName")
                    or sat_padrao or item.get("collection") or "desconhecido",
        "sensor": (instrumentos[0] if instrumentos else None)
                  or props.get("instrumentShortName") or sensor_padrao,
        "latitude": (min_lat + max_lat) / 2,
        "longitude": (min_lon + max_lon) / 2,
    }


def baixar_thumbnail(item) -> bytes | None:
    """Bytes do PNG do thumbnail, ou None (sem thumbnail, erro de rede ou não é imagem)."""
    asset = item.get("assets", {}).get("thumbnail")  # NUNCA "asset": é o .zip de ~1 GB
    if not (asset and asset.get("href")):
        return None
    try:
        resp = SESSAO.get(asset["href"], timeout=30)
        resp.raise_for_status()
    except requests.RequestException as erro:
        print(f"  falha ao baixar thumbnail de {item.get('id')}: {erro}")
        return None
    return resp.content if detectar_mime(resp.content) else None
```

8.4 core/dados/imagens.py

```
"""
imagens.py — Confere se uns bytes são mesmo uma imagem (usa a biblioteca Pillow).
Evita gravar no banco uma página de erro (HTML) como se fosse thumbnail.
"""

import io

from PIL import Image

FORMATOS = {"PNG": "image/png", "JPEG": "image/jpeg", "GIF": "image/gif", "WEBP":
"image/webp"}


def detectar_mime(conteudo: bytes) -> str | None:
    """Devolve 'image/png', 'image/jpeg', 'image/gif' ou 'image/webp'; None se não for
imagem."""
    try:
        with Image.open(io.BytesIO(conteudo)) as imagem:
            formato = imagem.format
            imagem.verify()
    except Exception:
        return None
    return FORMATOS.get(formato)
```

8.5 core/dados/popular_banco.py

```
"""
popular_banco.py — Enche o banco MySQL com 10000 thumbnails + metadados do INPE.

  python -m core.dados.popular_banco

Sem parâmetros. Baixa um thumbnail por vez; se a tabela já tiver registros, não faz nada.
"""

import pymysql
import requests
```

```
from core.dados import api_inpe, db

TOTAL = 10_000


def baixar_reais(conn, quantidade: int) -> int:
    """Busca cenas no INPE e baixa o thumbnail de cada uma, até juntar `quantidade`."""
    print("Buscando cenas no INPE (Cerrado Goiano)...")
    # pede 20% a mais: algumas cenas podem não ter thumbnail baixável
    cenas = api_inpe.buscar_itens(maximo_itens=quantidade + quantidade // 5 + 1)
    print(f"  {len(cenas)} cenas encontradas. Baixando thumbnails (um por vez)...")

    inseridas = 0
    for cena in cenas:
        if inseridas >= quantidade:
            break
        thumb = api_inpe.baixar_thumbnail(cena)
        if thumb is None:
            continue
        registro = api_inpe.extrair_metadados(cena)
        registro.update(tamanho_bytes=len(thumb), thumbnail_blob=thumb, origem="real")
        db.inserir(conn, registro)
        inseridas += 1
        if inseridas % 500 == 0:
            conn.commit()  # salva de tempos em tempos
        if inseridas % 100 == 0:
            print(f"  {inseridas}/{quantidade} thumbnails baixados")
    return inseridas


def _popular():
    with db.conectar() as conn:
        existentes = db.contar(conn)
        if existentes:
            print(f"O banco '{db.NOME_BANCO}' já tem {existentes} registros. Para recriar,
rode no phpMyAdmin (aba SQL): "
                  f"TRUNCATE TABLE metadados_imagem;")
            return
        try:
            total = baixar_reais(conn, TOTAL)
        except requests.RequestException as erro:
            raise SystemExit(f"Não foi possível acessar o INPE: {erro}")
        print(f"Pronto: {total} registros gravados.")
        if total < TOTAL:
            print(f"Atenção: faltaram {TOTAL - total} (cenas sem thumbnail).")


def main():
    try:
        _popular()
    except pymysql.err.OperationalError as erro:
        raise SystemExit(f"Não foi possível conectar ao MySQL: {erro}\n"
                         "Confira se o MySQL está ligado e os dados de conexão em
core/dados/db.py.")


if __name__ == "__main__":
    main()
```

8.6 core/dados/importar_sqlite.py

```
"""
importar_sqlite.py — Copia as linhas de um banco SQLite antigo para o MySQL.

Serve para reaproveitar o .sqlite que já foi baixado, sem baixar tudo do INPE de novo:

  python -m core.dados.importar_sqlite data/catalogo_cerrado.sqlite

Só copia se a tabela do MySQL estiver vazia (para não duplicar). Linhas sintéticas do banco
antigo são ignoradas.
"""

import os
import sqlite3
```

```
import sys

from core.dados import db

COLUNAS = ("nome_arquivo", "data_captura", "satelite", "sensor", "latitude", "longitude",
           "tamanho_bytes", "thumbnail_blob", "origem")


def main():
    if len(sys.argv) != 2:
        raise SystemExit("Uso: python -m core.dados.importar_sqlite
caminho/do/arquivo.sqlite")

    if not os.path.exists(sys.argv[1]):
        raise SystemExit(f"Arquivo não encontrado: {sys.argv[1]}")
    antigo = sqlite3.connect(sys.argv[1])
    linhas = antigo.execute(f"SELECT {', '.join(COLUNAS)} FROM metadados_imagem WHERE origem
IN ('real', 'manual') ORDER BY id").fetchall()
    antigo.close()

    with db.conectar() as conn:
        if db.contar(conn):
            raise SystemExit("A tabela do MySQL já tem registros; nada foi copiado.")
        for i, linha in enumerate(linhas, 1):
            db.inserir(conn, dict(zip(COLUNAS, linha)))
            if i % 500 == 0:
                conn.commit()
                print(f"  {i}/{len(linhas)}")
        print(f"Pronto: {len(linhas)} registros copiados para o MySQL.")


if __name__ == "__main__":
    main()
```

8.7 core/algoritmos/bubble_sort.py

```
"""
bubble_sort.py — Ordenação por bolha.

Complexidade: O(n²) em todos os casos, exceto com a otimização de parar
cedo quando não há mais trocas numa passada (melhor caso O(n) se a lista
já estiver ordenada).

Ideia: a cada passada, percorre a lista comparando pares vizinhos e troca
quando estão fora de ordem. O maior elemento "borbulha" até o fim a cada
passada completa.

Interface comum a todos os algoritmos deste pacote:
    sort(dados: list, key=lambda x: x) -> list

- NÃO modifica a lista original (retorna uma cópia ordenada) — isso é
  importante pro benchmark: cada algoritmo deve receber os mesmos dados
  de entrada, sem um "contaminar" o teste do outro.
- `key` extrai o valor usado na comparação (ex: lambda r: r['tamanho_bytes']),
  permitindo ordenar por qualquer campo do registro sem duplicar código.
"""


def sort(dados: list, key=lambda x: x) -> list:
    lista = dados.copy()
    n = len(lista)

    for i in range(n - 1):
        trocou = False
        for j in range(n - 1 - i):
            if key(lista[j]) > key(lista[j + 1]):
                lista[j], lista[j + 1] = lista[j + 1], lista[j]
                trocou = True
        if not trocou:
            break  # já está ordenada, não precisa das passadas restantes

    return lista
```

8.8 core/algoritmos/insertion_sort.py

```
"""
insertion_sort.py — Ordenação por inserção.

Complexidade: O(n²) no pior caso (lista invertida), mas O(n) no melhor
caso (lista já ordenada ou quase) — é o algoritmo que deve se destacar
no cenário "semi-ordenado" do benchmark, em contraste com o Bubble Sort.

Ideia: constrói a lista ordenada aos poucos, pegando um elemento de cada
vez e "inserindo" ele na posição correta entre os já ordenados — como
organizar cartas de baralho na mão.

Interface: igual à de bubble_sort.py (sort(dados, key) -> list, não mutante).
"""


def sort(dados: list, key=lambda x: x) -> list:
    lista = dados.copy()

    for i in range(1, len(lista)):
        atual = lista[i]
        chave_atual = key(atual)
        j = i - 1

        # desloca os elementos maiores que `atual` uma posição à direita
        while j >= 0 and key(lista[j]) > chave_atual:
            lista[j + 1] = lista[j]
            j -= 1

        lista[j + 1] = atual

    return lista
```

8.9 core/algoritmos/merge_sort.py

```
"""
merge_sort.py — Ordenação por intercalação (divisão e conquista).

Complexidade: O(n log n) garantido em qualquer caso (melhor, médio e
pior) — é o preço de ser mais previsível: sempre divide a lista ao meio,
sempre intercala. É estável (elementos com chaves iguais mantêm a ordem
relativa original), o que o conecta bem com o tema de "ordenação externa"
citado no enunciado — é a base de algoritmos que ordenam arquivos maiores
que a memória disponível, juntando pedaços já ordenados.

Interface: igual à de bubble_sort.py (sort(dados, key) -> list, não mutante).
"""


def sort(dados: list, key=lambda x: x) -> list:
    lista = dados.copy()

    if len(lista) <= 1:
        return lista

    meio = len(lista) // 2
    esquerda = sort(lista[:meio], key)
    direita = sort(lista[meio:], key)

    return _intercalar(esquerda, direita, key)


def _intercalar(esquerda: list, direita: list, key) -> list:
    """Combina duas listas já ordenadas em uma só, ordenada."""
    resultado = []
    i = j = 0

    while i < len(esquerda) and j < len(direita):
        # <= (não <) garante estabilidade: em caso de empate, o elemento
        # da esquerda (que veio antes na lista original) entra primeiro
        if key(esquerda[i]) <= key(direita[j]):
            resultado.append(esquerda[i])
            i += 1
```

```
        else:
            resultado.append(direita[j])
            j += 1

    resultado.extend(esquerda[i:])
    resultado.extend(direita[j:])
    return resultado
```

8.10 core/algoritmos/quick_sort.py

```
"""
quick_sort.py — Ordenação rápida (divisão e conquista, in-place por natureza).

Complexidade: O(n log n) em média — na prática costuma ser o mais rápido
dos comparativos — mas O(n²) no pior caso, que acontece justamente quando
o pivô escolhido é sempre o menor ou maior valor (ex: lista já ordenada
com pivô fixo no primeiro elemento). Por isso usamos um pivô pelo meio,
que mitiga (mas não elimina) esse risco nos cenários de teste.

Ideia: escolhe um "pivô", particiona a lista em (menores, iguais, maiores)
em relação a ele, e ordena cada partição recursivamente.

Interface: igual à de bubble_sort.py (sort(dados, key) -> list, não mutante).
Aqui a não-mutação é natural: cada chamada recursiva já cria novas listas
(menores/iguais/maiores), então o `dados` original nunca é tocado.
"""


def sort(dados: list, key=lambda x: x) -> list:
    if len(dados) <= 1:
        return dados.copy()

    pivo = dados[len(dados) // 2]
    chave_pivo = key(pivo)

    menores = [x for x in dados if key(x) < chave_pivo]
    iguais = [x for x in dados if key(x) == chave_pivo]
    maiores = [x for x in dados if key(x) > chave_pivo]

    return sort(menores, key) + iguais + sort(maiores, key)
```

8.11 benchmark/medir_desempenho.py

```
"""
medir_desempenho.py — Benchmark dos quatro algoritmos (monografia, seção Resultados).

Gera registros com chave tamanho_bytes aleatória (dados internos), prepara os cenários
aleatório / semiordenado / ordenado / invertido, mede só o sort() com time.perf_counter
e grava benchmark/resultados.json. Rode na raiz do projeto:  python -m
benchmark.medir_desempenho
"""
import sys, time, random, json, statistics, os
from core.algoritmos import bubble_sort, insertion_sort, merge_sort, quick_sort
ALG = {"Bubble Sort": bubble_sort, "Insertion Sort": insertion_sort, "Merge Sort": merge_sort,
"Quick Sort": quick_sort}
random.seed(2026)
# registros sintéticos com o mesmo formato do banco; chave = tamanho_bytes (PNG de miniatura ~
5 KB a 400 KB)
base = [{"id": i, "tamanho_bytes": random.randint(5_000, 400_000)} for i in range(1, 10001)]
chave = lambda r: r["tamanho_bytes"]
def preparar(n, cen):
    a = random.sample(base, n)
    if cen == "ordenado": a.sort(key=chave)
    elif cen == "invertido": a.sort(key=chave, reverse=True)
    elif cen == "semi":
        a.sort(key=chave)
        for _ in range(max(1, n // 20)):   # 5% de trocas aleatórias
            i, j = random.randrange(n), random.randrange(n); a[i], a[j] = a[j], a[i]
    return a
res = []
for cen in ["aleatorio", "semi", "ordenado", "invertido"]:
    for n in [100, 500, 1000, 2500, 5000, 10000]:
        reps = 5 if n <= 1000 else (3 if n <= 5000 else 2)
```

```
        dados = [preparar(n, cen) for _ in range(reps)]
        for nome, mod in ALG.items():
            ts = []
            for d in dados:
                t = time.perf_counter(); out = mod.sort(d, key=chave);
ts.append(time.perf_counter() - t)
                assert [chave(x) for x in out] == sorted(chave(x) for x in d)
            res.append({"cenario": cen, "n": n, "algoritmo": nome, "t": statistics.median(ts),
"reps": reps})
            print(cen, n, nome, round(res[-1]["t"], 6), flush=True)
json.dump(res, open(os.path.join(os.path.dirname(__file__), "resultados.json"), "w"),
indent=1)
print("FIM")
```

8.12 static/ordenacao.html

```
<!doctype html>
<html lang="pt-br">
<head>
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>Ordenação</title>
  <link rel="stylesheet" href="/static/estilo.css">
</head>
<body>
  <nav>
    <a href="/ordenacao" class="atual">Ordenação</a>
    <a href="/crud">CRUD</a>
    <a href="/comparador">Comparador</a>
  </nav>

  <h1>Ordenação com um algoritmo</h1>

  <form id="form">
    <label>Ordenar por
      <select id="campo">
        <option value="tamanho_bytes">Tamanho (bytes)</option>
        <option value="data_captura">Data de captura</option>
        <option value="nome_arquivo">Nome</option>
        <option value="satelite">Satélite</option>
        <option value="id">ID (ordem de inserção)</option>
      </select>
    </label>
    <label>Algoritmo
      <select id="algoritmo">
        <option value="quick">Quick Sort</option>
        <option value="merge">Merge Sort</option>
        <option value="insertion">Insertion Sort</option>
        <option value="bubble">Bubble Sort</option>
      </select>
    </label>
    <button type="submit" id="botao">Ordenar</button>
  </form>

  <div class="caixa">
    <div>Cronômetro ao vivo: <span class="grande" id="ao-vivo">0.00 s</span></div>
    <div>Tempo só da ordenação (sort): <span class="grande" id="tempo-sort">—</span></div>
  </div>

  <p id="status"></p>

  <table>
    <thead>
      <tr><th>Miniatura</th><th>ID</th><th>Nome</th><th>Satélite</th><th>Sensor</
th><th>Data</th><th>Tamanho (bytes)</th><th>Origem</th></tr>
    </thead>
    <tbody id="corpo"></tbody>
  </table>

  <div class="paginacao">
    <button id="anterior" disabled>Anterior</button>
    <span id="pagina-info"></span>
    <button id="proxima" disabled>Próxima</button>
  </div>
```

```
  <script src="/static/ampliar.js"></script>
  <script>
    const POR_PAGINA = 20;
    let itens = [];
    let pagina = 1;
    const $ = (id) => document.getElementById(id);

    function formatarTempo(s) {
      if (s < 0.001) return (s * 1e6).toFixed(0) + " µs";
      if (s < 1) return (s * 1000).toFixed(2) + " ms";
      return s.toFixed(3) + " s";
    }

    function celula(texto) {
      const td = document.createElement("td");
      td.textContent = texto ?? "—";
      return td;
    }

    function desenharPagina() {
      const totalPaginas = Math.max(1, Math.ceil(itens.length / POR_PAGINA));
      const linhas = itens.slice((pagina - 1) * POR_PAGINA, pagina * POR_PAGINA).map((item) =>
{
        const tr = document.createElement("tr");
        const tdImg = document.createElement("td");
        const img = document.createElement("img");
        img.src = `/api/imagens/${item.id}/thumbnail`;
        img.loading = "lazy";
        img.onerror = () => tdImg.replaceChildren("sem imagem");
        tdImg.appendChild(img);
        tr.append(tdImg, celula(item.id), celula(item.nome_arquivo), celula(item.satelite),
celula(item.sensor),
                  celula(item.data_captura), celula(item.tamanho_bytes), celula(item.origem));
        return tr;
      });
      $("corpo").replaceChildren(...linhas);
      $("pagina-info").textContent = `Página ${pagina} de ${totalPaginas}`;
      $("anterior").disabled = pagina <= 1;
      $("proxima").disabled = pagina >= totalPaginas;
    }

    $("form").addEventListener("submit", async (e) => {
      e.preventDefault();
      const inicio = performance.now();
      $("botao").disabled = true;
      $("status").textContent = "Ordenando...";
      $("status").className = "";
      $("tempo-sort").textContent = "—";
      const relogio = setInterval(() => {
        $("ao-vivo").textContent = ((performance.now() - inicio) / 1000).toFixed(2) + " s";
      }, 50);

      const params = new URLSearchParams({
        algoritmo: $("algoritmo").value, campo: $("campo").value,
      });
      try {
        const resposta = await fetch("/api/ordenar?" + params);
        const dados = await resposta.json();
        if (!resposta.ok) throw new Error(dados.erro || "erro no servidor");
        itens = dados.itens;
        pagina = 1;
        $("tempo-sort").textContent = formatarTempo(dados.tempo_segundos);
        $("status").textContent = `${dados.quantidade} registros ordenados com $
{dados.algoritmo}.`;
        desenharPagina();
      } catch (erro) {
        $("status").textContent = "Erro: " + erro.message;
        $("status").className = "erro";
      } finally {
        clearInterval(relogio);
        $("ao-vivo").textContent = ((performance.now() - inicio) / 1000).toFixed(2) + " s";
        $("botao").disabled = false;
      }
    });

    $("anterior").addEventListener("click", () => { pagina--; desenharPagina(); });
    $("proxima").addEventListener("click", () => { pagina++; desenharPagina(); });
```

```
  </script>
</body>
</html>
```

8.13 static/crud.html

```
<!doctype html>
<html lang="pt-br">
<head>
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>CRUD</title>
  <link rel="stylesheet" href="/static/estilo.css">
</head>
<body>
  <nav>
    <a href="/ordenacao">Ordenação</a>
    <a href="/crud" class="atual">CRUD</a>
    <a href="/comparador">Comparador</a>
  </nav>

  <h1>CRUD de imagens</h1>

  <form id="form">
    <label>Nome do arquivo *<input id="nome_arquivo" name="nome_arquivo" required></label>
    <label>Data de captura<input id="data_captura" name="data_captura" placeholder="2023-05-
12T13:40:00Z"></label>
    <label>Satélite<input id="satelite" name="satelite"></label>
    <label>Sensor<input id="sensor" name="sensor"></label>
    <label>Latitude<input id="latitude" name="latitude" type="number" step="any"></label>
    <label>Longitude<input id="longitude" name="longitude" type="number" step="any"></label>
    <label id="rotulo-imagem"><span id="texto-imagem">Imagem *</span><input id="imagem"
name="imagem" type="file" accept="image/*" required></label>
    <button type="submit" id="botao">Cadastrar</button>
    <button type="button" id="cancelar" hidden>Cancelar edição</button>
  </form>
  <p id="status"></p>

  <form id="busca">
    <label>Buscar por ID<input id="busca-id" type="number" min="1" step="1"></label>
    <button type="submit">Buscar</button>
    <button type="button" id="limpar-busca">Limpar busca</button>
  </form>

  <table>
    <thead>
      <tr><th>Miniatura</th><th>ID</th><th>Nome</th><th>Satélite</th><th>Sensor</
th><th>Data</th><th>Lat</th><th>Lon</th><th>Bytes</th><th>Origem</th><th>Ações</th></tr>
    </thead>
    <tbody id="corpo"></tbody>
  </table>

  <div class="paginacao">
    <button id="anterior">Anterior</button>
    <span id="pagina-info"></span>
    <button id="proxima">Próxima</button>
  </div>

  <script src="/static/ampliar.js"></script>
  <script>
    const $ = (id) => document.getElementById(id);
    const CAMPOS = ["nome_arquivo", "data_captura", "satelite", "sensor", "latitude",
"longitude"];
    let pagina = 1;
    let totalPaginas = 1;
    let editandoId = null;
    let buscaId = "";          // se preenchido, a tabela mostra só esse registro
    let versao = Date.now();   // muda ao editar: força o navegador a recarregar as miniaturas

    function mensagem(texto, erro = false) {
      $("status").textContent = texto;
      $("status").className = erro ? "erro" : "";
    }

    function celula(texto) {
```

```
      const td = document.createElement("td");
      td.textContent = texto ?? "—";
      return td;
    }

    function botao(texto, aoClicar) {
      const b = document.createElement("button");
      b.type = "button";
      b.textContent = texto;
      b.addEventListener("click", aoClicar);
      return b;
    }

    async function carregar() {
      const resposta = await fetch(buscaId ? "/api/registros?id=" +
encodeURIComponent(buscaId) : "/api/registros?pagina=" + pagina);
      const dados = await resposta.json();
      if (!resposta.ok) { mensagem("Erro: " + dados.erro, true); return; }
      totalPaginas = dados.total_paginas;
      if (pagina > totalPaginas) { pagina = totalPaginas; return carregar(); }

      const linhas = dados.itens.map((item) => {
        const tr = document.createElement("tr");
        const tdImg = document.createElement("td");
        const img = document.createElement("img");
        img.src = `/api/imagens/${item.id}/thumbnail?v=${versao}`;
        img.loading = "lazy";
        img.onerror = () => tdImg.replaceChildren("sem imagem");
        tdImg.appendChild(img);

        const tdAcoes = document.createElement("td");
        tdAcoes.append(botao("Editar", () => iniciarEdicao(item)), " ", botao("Excluir", () =>
excluir(item)));

        tr.append(tdImg, celula(item.id), celula(item.nome_arquivo), celula(item.satelite),
celula(item.sensor),
                  celula(item.data_captura), celula(item.latitude), celula(item.longitude),
                  celula(item.tamanho_bytes), celula(item.origem), tdAcoes);
        return tr;
      });
      $("corpo").replaceChildren(...linhas);
      $("pagina-info").textContent = !buscaId ? `Página ${pagina} de ${totalPaginas} ($
{dados.total} registros)`
        : dados.total ? `Resultado da busca pelo ID ${buscaId}` : `Nenhum registro com o ID $
{buscaId}`;
      $("anterior").disabled = pagina <= 1;
      $("proxima").disabled = pagina >= totalPaginas;
    }

    function iniciarEdicao(item) {
      editandoId = item.id;
      CAMPOS.forEach((c) => { $(c).value = item[c] ?? ""; });
      $("texto-imagem").textContent = "Nova imagem (opcional)";   // sem escolher, a imagem
atual continua
      $("imagem").required = false;
      $("botao").textContent = `Salvar alterações (id ${item.id})`;
      $("cancelar").hidden = false;
      window.scrollTo(0, 0);
    }

    function sairDaEdicao() {
      editandoId = null;
      $("form").reset();
      $("texto-imagem").textContent = "Imagem *";
      $("imagem").required = true;
      $("botao").textContent = "Cadastrar";
      $("cancelar").hidden = true;
    }

    async function excluir(item) {
      if (!confirm(`Excluir o registro ${item.id} (${item.nome_arquivo})?`)) return;
      const resposta = await fetch("/api/registros/" + item.id, { method: "DELETE" });
      const dados = await resposta.json();
      mensagem(resposta.ok ? `Registro ${item.id} excluído.` : dados.erro, !resposta.ok);
      carregar();
    }
```

```
    $("form").addEventListener("submit", async (e) => {
      e.preventDefault();
      let resposta;
      if (editandoId === null) {
        resposta = await fetch("/api/registros", { method: "POST", body: new FormData($
("form")) });
      } else {
        resposta = await fetch("/api/registros/" + editandoId, { method: "PUT", body: new
FormData($("form")) });
      }
      const dados = await resposta.json();
      if (!resposta.ok) { mensagem("Erro: " + dados.erro, true); return; }
      const cadastro = editandoId === null;
      mensagem(cadastro ? `Cadastrado com id ${dados.id}.` : `Registro ${dados.id}
atualizado.`);
      sairDaEdicao();
      if (cadastro) { pagina = 1; buscaId = ""; $("busca-id").value = ""; }   // novo registro
aparece no topo da lista
      versao = Date.now();
      carregar();
    });

    $("busca").addEventListener("submit", (e) => {
      e.preventDefault();
      buscaId = $("busca-id").value.trim();
      pagina = 1;
      carregar();
    });
    $("limpar-busca").addEventListener("click", () => {
      $("busca-id").value = "";
      buscaId = "";
      pagina = 1;
      carregar();
    });
    $("cancelar").addEventListener("click", sairDaEdicao);
    $("anterior").addEventListener("click", () => { pagina--; carregar(); });
    $("proxima").addEventListener("click", () => { pagina++; carregar(); });
    carregar();
  </script>
</body>
</html>
```

8.14 static/comparador.html

```
<!doctype html>
<html lang="pt-br">
<head>
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>Comparador</title>
  <link rel="stylesheet" href="/static/estilo.css">
</head>
<body>
  <nav>
    <a href="/ordenacao">Ordenação</a>
    <a href="/crud">CRUD</a>
    <a href="/comparador" class="atual">Comparador</a>
  </nav>

  <h1>Comparador de algoritmos</h1>
  <p class="pequeno" id="resumo"></p>

  <form id="form">
    <label>Quantidade de registros
      <input type="number" id="quantidade" value="1000" min="1">
    </label>
    <label>Ordenar por
      <select id="campo">
        <option value="tamanho_bytes">Tamanho (bytes)</option>
        <option value="data_captura">Data de captura</option>
        <option value="nome_arquivo">Nome</option>
        <option value="satelite">Satélite</option>
        <option value="id">ID (ordem de inserção)</option>
      </select>
    </label>
```

```
    <label>Dados de entrada
      <select id="cenario">
        <option value="aleatorio">Aleatórios (padrão)</option>
        <option value="ordenado">Pré-ordenados</option>
        <option value="invertido">Invertidos</option>
      </select>
    </label>
    <button type="submit" id="botao">Comparar</button>
  </form>

  <p id="status"></p>
  <div class="caixa" id="resultados" hidden></div>

  <script>
    const $ = (id) => document.getElementById(id);

    fetch("/api/resumo").then((r) => r.json()).then((d) => {
      $("resumo").textContent = `Registros no banco: ${d.total}`;
      $("quantidade").max = d.total;
    });

    $("form").addEventListener("submit", async (e) => {
      e.preventDefault();
      $("botao").disabled = true;
      $("resultados").hidden = true;
      $("status").className = "";
      $("status").textContent = "Comparando... (aguarde)";

      const params = new URLSearchParams({
        quantidade: $("quantidade").value, campo: $("campo").value, cenario: $
("cenario").value,
      });
      try {
        const resposta = await fetch("/api/comparar?" + params);
        const dados = await resposta.json();
        if (!resposta.ok) throw new Error(dados.erro || "erro no servidor");

        const linhas = dados.resultados.map((r) => {
          const div = document.createElement("div");
          div.className = "resultado";
          div.textContent = `${r.algoritmo}: ${r.tempo_segundos.toFixed(6)} s`;
          return div;
        });
        $("resultados").replaceChildren(...linhas);
        $("resultados").hidden = false;
        $("status").textContent = `${dados.quantidade} registros · campo: ${dados.campo} ·
dados: ${dados.cenario}`;
      } catch (erro) {
        $("status").textContent = "Erro: " + erro.message;
        $("status").className = "erro";
      } finally {
        $("botao").disabled = false;
      }
    });
  </script>
</body>
</html>
```

8.15 static/ampliar.js

```
// Clique em qualquer miniatura (imagem dentro de uma tabela) para ver a imagem grande.
// Clique em qualquer lugar ou tecle Esc para fechar.
(function () {
  const fundo = document.createElement("div");
  fundo.id = "ampliada";
  fundo.hidden = true;
  const grande = document.createElement("img");
  fundo.appendChild(grande);
  document.body.appendChild(fundo);

  function fechar() {
    fundo.hidden = true;
    grande.removeAttribute("src");
  }
```

```
  document.addEventListener("click", (e) => {
    const miniatura = e.target.closest("td img");
    if (miniatura) {
      grande.src = miniatura.src;
      fundo.hidden = false;
    } else if (!fundo.hidden) {
      fechar();
    }
  });
  document.addEventListener("keydown", (e) => { if (e.key === "Escape") fechar(); });
})();
```

8.16 static/estilo.css

```
body { font-family: Arial, sans-serif; max-width: 1000px; margin: 0 auto; padding: 16px;
color: #222; background: #fafafa; }
nav { display: flex; gap: 16px; padding-bottom: 8px; border-bottom: 2px solid #ccc; margin-
bottom: 16px; }
nav a { text-decoration: none; color: #0645ad; font-weight: bold; }
nav a.atual { color: #222; border-bottom: 3px solid #222; }
h1 { font-size: 1.4rem; margin: 0 0 12px; }
form { display: flex; flex-wrap: wrap; gap: 12px; align-items: end; margin-bottom: 16px; }
label { display: flex; flex-direction: column; font-size: 0.85rem; gap: 4px; }
input, select, button { padding: 6px 8px; font-size: 0.95rem; }
button { cursor: pointer; }
button:disabled { cursor: wait; opacity: 0.6; }
table { border-collapse: collapse; width: 100%; background: #fff; }
th, td { border: 1px solid #ccc; padding: 4px 8px; text-align: left; font-size: 0.85rem;
vertical-align: middle; }
th { background: #eee; }
td img { display: block; width: 64px; height: 64px; object-fit: cover; background: #ddd;
cursor: zoom-in; }
#ampliada { position: fixed; inset: 0; z-index: 10; display: flex; align-items: center;
justify-content: center;
            background: rgba(0, 0, 0, 0.85); cursor: zoom-out; }
#ampliada img { width: min(90vw, 90vh); height: min(90vw, 90vh); object-fit: contain; }
.caixa { background: #fff; border: 1px solid #ccc; padding: 10px 14px; margin-bottom: 12px; }
.grande { font-size: 1.8rem; font-family: monospace; font-weight: bold; }
.pequeno { font-size: 0.8rem; color: #555; }
.erro { color: #b00020; }
.paginacao { display: flex; gap: 12px; align-items: center; margin: 12px 0; }
.resultado { font-family: monospace; font-size: 1.05rem; margin: 4px 0; }
[hidden] { display: none !important; }
```

8.17 requirements.txt

```
flask>=3.0
requests>=2.31
pillow>=10.0
pymysql>=1.1
```

9 FICHA DE ATIVIDADES PRÁTICAS SUPERVISIONADAS

[Anexar aqui a ficha padrão de Atividades Práticas Supervisionadas, preenchida individualmente por cada aluno, com a cronologia dos itens realizados.]